

Rapport de Projet de Fin d'Études

Observabilité pilotée par IA et analyse des causes racines

*Conception et mise en œuvre de la plateforme d'observabilité
CIRES*

Lina Laaraich

CIRES Technologies / Tanger Med — Digital Factory Observability Service

Mai 2026

Version française du rapport, préparée pour la soutenance devant le jury francophone.

Résumé

Ce mémoire rend compte de la conception et de la mise en œuvre de la *plateforme d’observabilité CIREs*, une pile d’observabilité de bout en bout pour la production, enrichie d’une couche d’analyse des causes racines (RCA, *Root-Cause Analysis*) pilotée par intelligence artificielle. La plateforme a été construite entre février et mai 2026 chez CIREs Technologies (Tanger Med, Digital Factory Observability Service) au fil d’une séquence de sprints. Les sprints 0 à 3 ont été clôturés pendant la période du projet ; le sprint 4 est actuellement en cours (huit user stories livrées dès son premier jour opérationnel) et le périmètre du sprint 5 est consigné au chapitre 12 sous forme de plan prospectif qui étend la plateforme depuis un banc de test mono-applicatif vers un scénario microservice multi-applicatif assorti d’une détection hiérarchique d’anomalies.

Le travail répond à un problème opérationnel mis au jour pendant la phase de découverte : les incidents survenant dans l’environnement de production cible étaient détectés *après coup* — par les clients eux-mêmes ou lors d’une revue rétrospective des journaux — plutôt que par le système de supervision lui-même. La réponse de la plateforme prend la forme d’une architecture à deux couches : une fondation d’observabilité classique (Prometheus, Loki, Jaeger, Grafana, OpenTelemetry) provisionnée par Ansible sur AWS, surmontée d’un service de tri basé sur FastAPI qui consomme les alertes via les webhooks Grafana, collecte un contexte transversal aux trois piliers au travers de cinq serveurs MCP (Model-Context-Protocol, protocole de mise en contexte du modèle) et produit des analyses structurées des causes racines à l’aide d’un grand modèle de langage local. La couche de modèle a migré le 2026-05-21 depuis `qwen2.5:7b-instruct` sur un GPU de portable rattaché par Tailscale vers `qwen2.5:14b-instruct` sur une instance AWS dédiée `g5.xlarge` (A10G) en `us-west-2`, contrôlée par une passerelle Lambda d’extinction automatique ; le *runner* portable est conservé en repli. La latence médiane de bout en bout du pipeline est passée d’environ cinq minutes sur le portable à trente-huit secondes sur l’hôte GPU dans le *cloud*.

La mise en œuvre introduit plusieurs mécanismes inédits pour garder la sortie du LLM (*Large Language Model*, grand modèle de langage) digne de confiance sur le plan opérationnel : une bibliothèque de quatorze archétypes canoniques de RCA injectés en tant qu’échafaudage structurel avant le bloc de preuves ; une liste noire d’hallucinations indexée par alerte ; une paire de validateurs détectant respectivement le raisonnement de surface et le « menu d’hypothèses » ; un bridage de la confiance qui retire les actions suggérées lorsque les signaux de confiance sont faibles et émet à la place des *étapes de diagnostic* en lecture seule ; une unique reprise à action contenue (*bounded agency*) limitée à un seul appel MCP ; et une couche de retour d’information en boucle fermée qui transforme les corrections de l’opérateur en signal de routage. Un *invariant d’accès aux données via MCP uniquement* est imposé comme règle à l’échelle du projet : chaque donnée que voit le LLM doit transiter par un pont MCP, jamais par des imports en mémoire ou des requêtes directes à la base de données ou à Prometheus.

Le mémoire présente l’architecture, l’historique de mise en œuvre sprint par sprint, une étude de cas approfondie de l’incident `0b215ef3` qui a déclenché l’épopée du pare-feu anti-hallucination, une évaluation contre une grille de qualité RCA à quatre axes, ainsi qu’un catalogue des vingt-cinq décisions durables de conception (D1–D25, les deux dernières actant la migration GPU et le verrouillage d’accès qui l’a accompagnée) consignées pendant la construction. La suite de tests du service de tri affichait 296/296 au vert le premier jour du sprint 4 (2026-05-25), le lint d’invariant MCP-seul était propre et le *runner* opérationnel était resté en service sans interruption au travers des deux générations de *runner*. Les deux derniers chapitres définissent le périmètre du sprint 4 en cours ainsi que le plan du sprint 5, comprenant la collecte agentique

itérative, la génération augmentée par récupération (RAG) sur un corpus curé alimenté par le retour d'information de l'opérateur, un banc de test microservice multi-applicatif et la conception d'une détection d'anomalies hiérarchique à trois niveaux fondée sur Drain3.

Mots-clés : observabilité, SRE, AIOps, analyse des causes racines, MCP, Prometheus, Loki, Jaeger, Grafana, grands modèles de langage, Ollama, qwen2.5, Ansible, Terraform, k3s.

Table des matières

Résumé	1
1 Introduction	9
1.1 Contexte et motivation	9
1.2 Énoncé du problème	9
1.3 Objectif et objectifs spécifiques	9
1.4 Périmètre et limites	10
1.5 Structure du mémoire	10
2 État de l’art et travaux connexes	12
2.1 Les trois piliers de l’observabilité	12
2.2 La pratique SRE et les signaux d’or	12
2.3 Prometheus, Loki, Jaeger, Grafana	13
2.4 Drain3 et le regroupement de modèles de journaux	13
2.5 Les grands modèles de langage pour l’exploitation informatique	13
2.6 Le protocole Model-Context-Protocol	14
2.7 Plateformes apparentées et travaux antérieurs	15
3 Découverte et recueil des besoins (sprint 0)	16
3.1 Démarche	16
3.2 Constats issus des entretiens	16
3.3 Exploration de l’espace des solutions	17
3.4 Exigences fonctionnelles	17
3.5 Exigences non fonctionnelles	17
4 Architecture du système	19
4.1 Vue d’ensemble en couches	19
4.2 Topologie d’exécution	19
4.3 Flux de données : de l’alerte à la RCA	20
4.4 L’invariant d’accès aux données via MCP uniquement	20
5 Sprint 1 — Fondation d’observabilité	26
5.1 Objectif du sprint et topologie	26
5.2 Disposition des rôles Ansible	26
5.3 Télémétrie à trois piliers	26
5.4 Grafana et alerting initial	26
5.5 Résultat du sprint 1	27
6 Sprint 2 — Migration AWS et MVP RCA par IA	28
6.1 Objectif du sprint	28
6.2 Épopée 1 — Infrastructure AWS	28
6.3 Épopée 2 — MVP du système RCA par IA	28
6.4 Épopée 3 — Migration Kubernetes	29
6.5 Épopée 4 — Renforcement du tri par IA	29
6.6 Épopée 5 — User stories à coloration UEBA (post-sprint)	29
6.7 Résultat du sprint 2	30

7	Sprint 3 — Qualité, retour d’information et pare-feu anti-hallucination	31
7.1	Forme du sprint	31
7.2	La grille de qualité RCA à 4 axes	31
7.3	La bibliothèque d’exemples (D17)	32
7.4	Seuils adaptatifs (D18)	32
7.5	Philosophie de la prose RCA (D19)	32
7.6	Retour d’information en boucle fermée (D20)	33
7.7	Barrières de récurrence (D21)	33
7.8	Arbres de modèles Drain3 par service (D23)	33
7.9	L’épopée du pare-feu anti-hallucination (Épopée 4)	34
7.10	La fenêtre d’investigation de 20 jours (2026-04-30 au 2026-05-19)	35
7.11	Rafraîchissement documentaire	35
7.12	La reprise à action contenue	36
8	Étude de cas : l’incident 0b215ef3	37
8.1	L’alerte	37
8.2	La mauvaise RCA	37
8.3	Analyse <i>forensic</i> : trois violations de l’invariant MCP	37
8.4	La réponse échelonnée	38
8.5	Leçons retenues	38
9	Évaluation	40
9.1	Progression de la couverture de tests	40
9.2	Qualité RCA sur l’échantillon de 28 décisions	40
9.3	Audit critique de la sortie RCA — 12 cas représentatifs (2026-05-21)	40
9.3.1	Grille d’audit	41
9.3.2	Taux de réussite par critère	41
9.3.3	Taxonomie des modes de défaillance avec cas spécifiques	41
9.3.4	Ce qui fonctionne	43
9.3.5	Verdict global et alignement sur le sprint 4	43
9.4	Résultats des runs de chaos	43
9.5	Cohérence du corpus d’audit	44
9.6	Budget de latence	45
9.7	Régime opérationnel permanent	45
10	Décisions de conception et justifications	46
10.1	Trois décisions en profondeur	47
10.1.1	D17 — Pourquoi les exemplaires fonctionnent là où la récupération a échoué	47
10.1.2	L’invariant MCP-seul comme règle à l’échelle du projet	48
10.1.3	L’action contenue plutôt que la collecte agentique complète	48
11	Sprint 4 — En cours	49
11.1	Classement des candidats du sprint 4 (report)	49
11.2	Livré avant la clôture du jour 1 (huit user stories)	52
11.2.1	D24 + D25 — Migration GPU <i>cloud</i> (livrée 2026-05-21)	52
11.2.2	SF-6 — Refonte du courriel (livrée 2026-05-23, commit f6d0a1b)	53
11.2.3	SF-7 — Page de retour d’opérateur (livrée 2026-05-23, commit 10ebd4e)	53
11.2.4	SF-4 — Regroupement par même alerte sur le tableau de bord (livré 2026-05-23, commit 7d0adeb)	54
11.2.5	Phase 2.1 — Route de la page de détail (livrée 2026-05-22, commits 9b61e74 + a904eda)	54

11.2.6 DA-5 + DA-4 — Déduplication par famille d’alertes + empreintes drain3 par hachage de contenu (livré 2026-05-25, commit b8fa2c8)	54
11.2.7 Observation de livraison du jour 1	54
11.3 En file d’attente pour le reste du sprint 4	54
11.4 Tier 3, RAG curé et palier résilience (reportés au sprint 5)	55
11.5 Ancre de section héritée pour la migration GPU	56
12 Sprint 5 — Planifié (2026-06-06 au 2026-06-17)	57
12.1 EPIC13 — Banc de test microservice + scénario canonique	57
12.2 EPIC14 — Entité incident + surfaces d’insights barre latérale	58
12.3 EPIC15 — Configuration opérateur + durcissement production	59
12.4 Nouvel élément de conception — S5-DRN-01 seuils Drain3 hiérarchiques	59
12.5 Forme de déploiement production — OTel à agent unique pour le NOC	61
12.6 Perspectives sprint 6+	61
13 Conclusion	63
A Inventaire des dépôts	64
B Glossaire	65
C Résumé de la chronologie des sprints	67
Références bibliographiques	68

Table des figures

4.1	Architecture en couches. Chaque couche ne consomme que la couche immédiatement inférieure ; l'invariant MCP-seul est la règle qui formalise ce principe pour la couche IA.	19
4.2	Phase 1 — fondation d'observabilité. L'instrumentation applicative (Spring Boot, Kong, MySQL, k3s) alimente les collecteurs (scrape Prometheus, OTel Collector avec son récepteur <code>filelog/containers</code> pour les journaux et son récepteur OTLP pour les traces) et le stockage (Prometheus TSDB, Loki, Jaeger). Grafana expose les données et achemine les alertes via le point de contact webhook de l'alerting unifié.	21
4.3	Phase 2 — couche de tri par IA. Les récepteurs de webhooks (Grafana, Drain3) alimentent le pipeline en dix étapes ; la collecte de contexte interroge en parallèle les cinq ponts MCP ; le LLM est invoqué une seule fois sur le bloc de preuves pré-collecté ; les validateurs, le bridage de confiance et la reprise à action contenue ferment la boucle avant que la RCA ne soit persistée, affichée sur le tableau de bord et envoyée comme courriel d'escalade.	22
4.4	Vue d'ensemble complète de la plateforme (phase 1 + phase 2) à échelle réduite, conservée pour la mise en correspondance avec les panneaux séparés ci-dessus. Une variante <i>de style designer</i> générée en matplotlib (<code>architecture-phase1-observability-v2.png</code>) est conservée dans le dépôt comme alternative stylistique.	22
4.5	Le pipeline de tri en dix étapes. Les étapes 6/6b/6c/6d forment ensemble le pare-feu anti-hallucination.	23
4.6	Le pipeline de tri en organigramme vertical. Une alerte descend depuis le récepteur de webhooks à travers les étapes de déduplication, de suppression et de barrière de récurrence, vers la collecte de contexte, l'appel LLM, la pile de validateurs et les étapes de persistance et de notification. Les deux sorties latérales précoces (supprimée avant LLM, écartée par le validateur) sont la défense de la conception contre les appels LLM gaspillés et les sorties opérationnellement non fiables. . .	24
4.7	L'invariant d'accès aux données via MCP uniquement représenté comme un diagramme de pare-feu. Le LLM ne voit que ce qui arrive par l'un des cinq ponts MCP ; les chemins en pointillés montrent les trois accès directs (en mémoire ou en HTTP direct) que l'incident <code>0b215ef3</code> a exposés et que l'épopée anti-hallucination du sprint 3 a refermés. Le lint d'intégration continue livré comme S3-HF-08 est la couche d'application mécanique.	25
9.1	La matrice de décision opérateur (verdict LLM $\times$ qualité RCA) $\rightarrow$ <code>action_taken</code> , avec la voie pré-LLM affichée séparément. La matrice encode la réponse de la plateforme à chaque combinaison de signal de confiance et de qualité ; la voie pré-LLM (supprimée par empreinte, supprimée par barrière de récurrence) est rendue sur une piste distincte parce que ces décisions sont prises avant qu'un appel LLM ne soit dispatché et n'ont donc pas de verdict à combiner avec le signal de qualité.	44
12.1	Feuille de route des sprints, du sprint 0 au sprint 6+. Chaque colonne porte la fenêtre du sprint et son résultat phare ; les jalons sous la chronologie sont les temps structurels du projet (énoncé du problème, fondation d'observabilité, MVP IA, pare-feu anti-hallucination, migration GPU, banc microservice, Drain3 hiérarchique). Le sprint 4 est ombré comme <i>en cours</i>	57

12.2 La conception sprint 5 des seuils Drain3 hiérarchiques — trois paliers (composant, application, système) qui se déclenchent indépendamment les uns des autres. Le déclenchement de Tier N ne supprime pas Tier $N+1$. Le panneau de droite annote le palier « intersection » rejeté (applications partageant une infrastructure), abandonné comme combinatoirement flou.	60
--	----

Liste des tableaux

6.1	User stories de l'Épopée 5 reportées au-delà de la clôture du sprint 2.	29
7.1	Résultats de la fenêtre d'investigation (du 2026-04-30 au 2026-05-19). Chaque candidat a été noté sur la grille de qualité RCA à 4 axes sur un échantillon de vingt-huit décisions.	35
9.1	Progression de la suite de tests du service de tri.	40
9.2	Qualité RCA de référence sur l'échantillon de 28 décisions (plus haut est meilleur). « Actionnable » est le score le plus bas en raison du comportement délibéré de retrait des actions lors du bridage (S3-HF-01), qui abaisse le score d'actionnabilité mais élève la confiance opérationnelle.	40
9.3	Taux de réussite par critère de l'audit critique RCA sur 12 cas échantillonnés. C4 (sans bruit) et C6 (étapes de diagnostic non modélisées) sont les deux pires dimensions ; C8 (pas d'hallucination de métrique tronquée) est la plus solide uniquement parce que l'artefact spécifique est rare dans le corpus, non parce que le garde-fou existe encore (P4 n'est pas livré).	42
10.1	Catalogue des décisions durables de conception D1–D25.	46
11.1	Candidats du sprint 4 au 2026-05-19, classés. La colonne « P-tag » associe chaque ligne aux constats P0–P11 de l'investigation du 2026-05-19 lorsque pertinent ; -- indique un élément de report sans constat d'investigation attaché.	49
A.1	Disposition des dépôts du projet (sept dépôts).	64
C.1	Chronologie des sprints, condensée. Sprints 0–3 clôturés ; sprint 4 en cours le 2026-05-25 ; sprint 5 scopé ; perspectives sprint 6+ consignées.	67

CHAPITRE 1

Introduction

1.1 Contexte et motivation

CIRES Technologies est la *digital factory* de Tanger Med, le principal complexe portuaire du nord du Maroc. La Digital Factory Observability Service est responsable de la santé opérationnelle d'un catalogue interne de services qui soutient la logistique portuaire, les intégrations clients et les flux administratifs. Pour prototyper une posture d'observabilité améliorée sans perturber les systèmes de production, le périmètre du projet a été délibérément défini autour d'une *pile web représentative* (un *back-end* Spring Boot, une passerelle d'API Kong, une base de données MySQL et un *front-end* React) déployée sur un petit parc de machines virtuelles sur site — environnement à échelle réduite destiné à prototyper l'architecture avant toute transition vers la production CIREs ; une migration parallèle du prototype vers AWS figurait déjà sur la feuille de route à moyen terme.

La supervision interne au démarrage du projet se limitait à des inspections ponctuelles de journaux et à un petit nombre de règles basées sur des seuils métriques. Sur le plan opérationnel, cela produisait une posture structurellement réactive : les pannes étaient détectées par les retours des clients ou lors d'une revue rétrospective des journaux, jamais par le système lui-même. L'asymétrie entre la vitesse de la défaillance (de la seconde à la minute) et la vitesse de détection humaine (de l'heure à la journée) définissait l'écart opérationnel que ce projet entend combler.

1.2 Énoncé du problème

Énoncé du problème (issu des recherches de terrain du sprint 0)

L'équipe a besoin d'un moyen de détecter précocement les problèmes de production — avant qu'ils n'atteignent un client et sans qu'un humain doive avoir les yeux rivés sur les journaux au bon moment. La détection doit aussi être *pré-investigée* : chaque alerte doit arriver avec une cause racine candidate, les preuves qui la soutiennent et une remédiation recommandée, de sorte que le premier geste de l'opérateur soit de valider ou d'invalidier plutôt que d'ouvrir une investigation à partir de zéro.

Cette formulation scinde le problème en deux moitiés distinctes. La première est conventionnelle : déployer une véritable pile d'observabilité afin que le système émette des signaux exploitables. La seconde est inédite : superposer à cette pile un système de raisonnement automatisé qui transforme les alertes en RCA structurées et actionnables sans solliciter l'attention humaine de l'astreinte. La plateforme met en œuvre les deux moitiés et les intègre de bout en bout.

1.3 Objectif et objectifs spécifiques

Objectif général. Concevoir, mettre en œuvre et valider une plateforme d'observabilité de forme proche de la production qui détecte, investigate et restitue les incidents de manière proactive, grâce à une couche de tri par IA s'appuyant sur une infrastructure *open source*, sans dépendance à des fournisseurs SaaS d'observabilité hébergés.

Objectifs spécifiques.

1. Établir une fondation d’observabilité à trois piliers (métriques, journaux, traces) pour la pile web cible, déployable depuis une infrastructure vierge au travers d’un unique *playbook* Ansible.
2. Migrer la charge de travail depuis les machines virtuelles sur site vers un environnement AWS géré par Terraform et exécuter la charge applicative sur Kubernetes (k3s) tout en maintenant la pile de supervision sur Docker Compose pour simplifier l’exploitation.
3. Mettre en œuvre un service de tri par IA qui consomme les webhooks Grafana et produit des analyses structurées des causes racines avec verdicts, preuves et actions de remédiation suggérées, persistées dans un magasin d’historique durable.
4. Faire transiter chaque source de télémétrie (Prometheus, Loki, Jaeger, historique de tri, événements de déploiement) derrière un serveur MCP et imposer un invariant d’accès aux données via MCP uniquement, de sorte que chaque donnée consommée par le LLM soit traçable via un pont.
5. Développer et évaluer des mécanismes de contrôle qualité qui gardent la sortie du LLM digne de confiance sur le plan opérationnel : injection d’exemplaires, liste noire d’hallucinations, validateurs de forme de la prose, bridage de la confiance et canal de retour d’information en boucle fermée avec l’opérateur.
6. Documenter le système de bout en bout à un niveau suffisant pour que l’équipe d’ingénierie puisse l’exploiter, l’étendre et l’auditer après la clôture du projet.

1.4 Périmètre et limites

Le projet inclut explicitement les composants ci-dessus dans son périmètre et en exclut explicitement les éléments suivants :

- Les fournisseurs SaaS d’observabilité hébergés (Datadog, New Relic, Honeycomb). Ils ont été chiffrés et étudiés pendant le sprint 0 ; le choix d’une infrastructure *open source* a été dicté par le budget, la souveraineté des données et les objectifs d’apprentissage.
- Le déploiement en production sur des systèmes réellement exposés au client. Le périmètre se limite à la pile web représentative utilisée comme substitut de développement (un dérivé du dépôt `mukundmadhav/react-springboot-mysql`), déployée sur une infrastructure dédiée avec une charge synthétique et une injection contrôlée de défaillances.
- La haute disponibilité multi-régions pour la pile d’observabilité elle-même. La plateforme s’exécute en `us-east-1` (ou en local sur site) et n’est pas multi-AZ.
- Un *runner* GPU dans le *cloud* pour la couche LLM. Cette piste était à l’origine envisagée comme candidate du sprint 4, abandonnée pour des raisons de coût le 2026-05-19, puis réintroduite et livrée le 2026-05-21 une fois le cadre de coût reconstruit autour d’une passerelle Lambda d’extinction automatique (voir §11.5).

1.5 Structure du mémoire

Le mémoire est organisé comme suit. Le chapitre 2 passe en revue les fondements techniques : pratique de l’observabilité, SRE, regroupement de modèles de journaux, grands modèles de langage pour l’exploitation informatique et spécification du protocole MCP. Le chapitre 3 traite

la phase de découverte du sprint 0, comprenant les résultats des entretiens et le cadrage qui a produit l'énoncé du problème ci-dessus. Le chapitre 4 présente l'architecture de la plateforme dans sa forme actuelle. Les chapitres 5–7 couvrent les trois sprints de mise en œuvre clôturés dans l'ordre chronologique. Le chapitre 8 fournit une étude de cas approfondie de l'incident 0b215ef3 qui a déclenché l'épopée du pare-feu anti-hallucination. Le chapitre 9 évalue la plateforme à l'aune d'une grille de qualité RCA à quatre axes, du banc d'injection de chaos et des cycles récurrents d'audit statique et en condition réelle. Le chapitre 10 catalogue les vingt-cinq décisions durables de conception (D1–D25) consignées pendant la construction et examine les trois plus marquantes en profondeur. Le chapitre 11 documente le travail du sprint 4 actuellement en cours (huit user stories livrées le jour 1, cinq programmées sur le reste du plan). Le chapitre 12 présente le plan du sprint 5 et les perspectives à plus long terme du sprint 6 et au-delà, en y incluant la nouvelle conception des *seuils Drain3 hiérarchiques*. Le chapitre 13 conclut.

CHAPITRE 2

État de l’art et travaux connexes

2.1 Les trois piliers de l’observabilité

La pratique moderne de l’observabilité identifie trois piliers primaires de la télémétrie : les *métriques*, les *journaux* et les *traces*. Les métriques sont des séries temporelles numériques décrivant l’état du système (utilisation CPU, débit de requêtes, taux d’erreurs, etc.), généralement collectées à intervalle fixe et stockées dans une base de données spécialisée comme Prometheus. Les journaux sont des événements textuels discrets émis par les services, idéalement avec des champs structurés, agrégés par une base de données de journaux comme Grafana Loki. Les traces sont les enregistrements distribués du parcours d’une requête à travers les services ; elles transportent un identifiant de trace qui relie les *spans* enregistrés par chaque saut en une structure unique à examiner. Le projet OpenTelemetry fournit le format réseau canonique et Jaeger est un *back-end* de stockage largement adopté.

Un système pleinement observable requiert les trois piliers ainsi que la capacité de naviguer entre eux : depuis une métrique anormale vers les journaux du service dont la métrique est anormale, et de ces journaux vers la trace qui transportait la requête à l’origine. La plateforme CIRES met en œuvre cette triangulation comme une préoccupation de premier rang ; l’étape de collecte de contexte du service de tri par IA fait explicitement appel aux trois piliers ainsi qu’à un quatrième signal, le regroupement de modèles de journaux fondé sur Drain3, avant tout raisonnement par le LLM.

2.2 La pratique SRE et les signaux d’or

La littérature de Google sur le *Site Reliability Engineering* (le *SRE Book* et le *SRE Workbook*) identifie quatre *signaux d’or* (*golden signals*) à l’aune desquels presque tout service en contact avec l’utilisateur final devrait être mesuré : **la latence, le trafic, les erreurs et la saturation**. L’ouvrage articule également deux idées qui ont influencé la taxonomie des alertes du projet :

- La distinction **symptômes vs. causes**. Les alertes doivent se déclencher sur des symptômes observables par l’utilisateur (latence élevée, taux d’erreurs élevé) plutôt que sur des causes sous-jacentes (CPU élevé, mémoire élevée). Les alertes basées sur les causes produisent un flot de faux positifs dans les systèmes en bonne santé ; les alertes basées sur les symptômes ne sollicitent l’opérateur que lorsque quelque chose est réellement cassé du point de vue de l’utilisateur.
- Le cadrage du **budget d’erreur**. Les objectifs de niveau de service (SLO) définissent un taux d’erreurs tolérable ; l’écart entre la fiabilité visée et 100 % de fiabilité constitue le budget d’erreur. Une équipe dans le rouge sur son budget d’erreur doit ralentir le développement de fonctionnalités et investir dans la fiabilité ; une équipe dans le vert a la permission explicite de prendre plus de risques.

La plateforme CIRES ne formalise pas, à ce stade, les SLO ou les budgets d’erreur, mais l’ensemble des règles d’alerte est délibérément structuré pour pencher du côté symptomatique — **HighP95Latency** sur la passerelle, **HighKongP95Latency** par *upstream*, **ErrorRate** sur la couche

service — avec les alertes en forme de causes (CPU, mémoire, disque) traitées comme signal diagnostique d'appoint plutôt que comme matière première à l'astreinte.

2.3 Prometheus, Loki, Jaeger, Grafana

La pile fondationnelle de la plateforme est la combinaison *open source* de fait pour l'observabilité moderne des infrastructures :

- **Prometheus** pour les métriques. Collecte (*pull*) à intervalle de 15 s. Spring Boot expose `/actuator/prometheus`, Kong expose son endpoint de statistiques, et au niveau hôte `node_exporter` et `cAdvisor` comblent les manques.
- **Loki** pour les journaux. Ingestion via le récepteur `filelog/containers` de l'OpenTelemetry Collector, qui suit les journaux de conteneurs Docker sous `/var/lib/docker/containers/**/*-json.log` et les transmet via l'exportateur natif `loki` du collecteur vers `loki:3100/loki/api/v1/push`. Loki stocke les flux de journaux indexés par étiquettes plutôt que par contenu plein texte, ce qui maintient les coûts de stockage faibles et s'aligne proprement sur le modèle d'étiquettes de Prometheus.
- **Jaeger** pour les traces. L'OpenTelemetry Collector reçoit les *spans* de l'agent Java OTEL injecté dans Spring Boot et du *plug-in* OTEL de Kong à la couche passerelle, puis les transmet à Jaeger. Les identifiants de trace se propagent de bout en bout.
- **Grafana** pour la visualisation et le système d'alertes. L'alerting unifié de Grafana a remplacé Alertmanager au début du projet (voir la décision D6 au chapitre 10) ; un unique point de contact `triage-webhook` achemine tous les événements d'alerte vers le service de tri par IA.

2.4 Drain3 et le regroupement de modèles de journaux

Les journaux de production réels contiennent un petit nombre de chaînes-modèles récurrentes et une grande masse de variations incidentes (horodatages, identifiants, valeurs de paramètres). L'algorithme Drain3 (une variante paramétrable de Drain) ingère les lignes de journal, remplace les parties variables par des substituts et regroupe les lignes par le modèle obtenu. Une fois le catalogue de modèles stabilisé, les modèles nouveaux deviennent un signal opérationnel fort : une ligne de journal dont le modèle n'a jamais été observé est, par construction, inhabituelle.

La plateforme CIREs intègre Drain3 comme quatrième signal aux côtés des métriques, des journaux et des traces. Un *poller* de fond lit les lignes de journal les plus récentes depuis Loki, les fait passer par un mineur de modèles par service et émet un webhook synthétique `Drain3AnomalyDetected` vers le service de tri chaque fois que le taux récent de nouveautés dépasse un seuil configurable. La charge utile du webhook embarque les chaînes-modèles nouvelles réelles ainsi qu'un échantillon des lignes sous-jacentes, de sorte que le LLM en aval voit non seulement un comptage mais aussi les preuves elles-mêmes (la décision D23 acte la séparation par service qui a résolu une classe de faux positifs introduits par un mineur global antérieur).

2.5 Les grands modèles de langage pour l'exploitation informatique

L'usage des grands modèles de langage pour les tâches d'exploitation informatique est un domaine actif parfois étiqueté *AIOps*. L'état de la pratique varie largement. Les fournisseurs

d’observabilité hébergés livrent des fonctionnalités à connotation IA depuis plusieurs années (Datadog Watchdog, New Relic AI, BubbleUp de Honeycomb, le résumé d’incident de Komodor), chacun avec un périmètre différent : détection d’anomalies, résumé d’alertes, corrélation entre signaux, reconstitution d’incidents.

Trois risques dominant lorsque un LLM est placé dans le chemin opérationnel :

1. **L’hallucination.** Le modèle peut produire avec assurance des affirmations qui ne découlent pas du bloc de preuves — valeurs métriques inventées, services qui n’existent pas, commandes `kubect1` pour des systèmes qui ne sont pas déployés sur Kubernetes.
2. **Le raisonnement de surface.** Le modèle peut reformuler l’alerte avec d’autres mots (« l’alerte indique une latence élevée » ou « sur la base de la valeur observée de 1,2 s ») sans identifier de cause.
3. **Les menus d’hypothèses.** Sous incertitude, le modèle peut émettre une liste de possibilités (« cela peut être une requête lente, un problème de pool de connexions ou une pause de garbage collection ») plutôt que de s’engager sur l’une d’elles en en nommant les preuves.

Un défi central d’ingénierie du projet a consisté à concevoir des mécanismes qui détectent et rejettent chacun de ces modes de défaillance sans rejeter les RCA réellement bonnes. Les chapitres 7 et 8 traitent ces mécanismes en profondeur.

2.6 Le protocole Model-Context-Protocol

Le *Model-Context-Protocol* (MCP) est une spécification ouverte, introduite en 2024 par Anthropic et désormais adoptée par toute une gamme d’outils orientés LLM, destinée à connecter les applications LLM à des sources de données et des outils externes. Un *serveur* MCP expose un petit nombre d’outils (typiquement en lecture seule) sur un transport JSON-RPC ; un *client* MCP découvre les outils, les invoque et réinjecte les résultats dans le contexte du modèle.

La plateforme CIRES utilise MCP non dans sa forme originelle « le LLM appelle directement les outils » mais dans un schéma plus contraint. Cinq serveurs MCP s’intercalent entre les sources de données et le service de tri :

- `prometheus-mcp:8091` — requêtes instantanées et plages contre Prometheus.
- `loki-mcp:8092` — requêtes de journaux contre Loki.
- `jaeger-mcp:8093` — consultations de traces contre Jaeger, plus un outil `get_trace` pour la récupération d’une trace complète.
- `rca-history-mcp:8094` — accès en lecture à l’historique RCA persistant (SQLite, sur le *runner* portable).
- `deploy-events-mcp:8095` — chronologie d’événements de déploiement issue des dépôts applicatifs.

Dans le chemin par défaut, le service de tri *lui-même* (et non le LLM) interroge ces serveurs MCP pendant la collecte de contexte et assemble un bloc de preuves rendu dans le *prompt* sous forme d’une section structurée **## Pre-Gathered Context**. Le LLM est invoqué une seule fois sur ce matériel pré-collecté. Seul le chemin de reprise à action contenue (chapitre 7, §7.12) donne au LLM un accès direct aux outils MCP, et un seul appel de ce type est autorisé par reprise.

Cette forme est délibérée : elle préserve les forces du LLM (génération à sortie structurée, synthèse narrative) tout en maintenant les chemins opérationnels (quelles requêtes lancer, quelles preuves collecter) sous contrôle déterministe. **L’invariant MCP-seul** — « chaque donnée que voit le LLM doit transiter par un pont MCP » — est le pare-feu anti-hallucination du projet dans sa formulation la plus générale.

2.7 Plateformes apparentées et travaux antérieurs

- **Datadog Watchdog** réalise une détection d’anomalies à travers les métriques et produit des résumés narratifs d’incidents. Il s’agit d’un système propriétaire fermé ; la construction *open source* de la plateforme CIRES permet à l’équipe d’inspecter et d’ajuster chaque couche.
- **New Relic Applied Intelligence (NRAI)** fournit une corrélation et une priorisation entre alertes. Son graphe d’« incident intelligence » est un précédent utile pour le *corrélateur d’incidents* (US-5.2) que la plateforme CIRES inscrit au périmètre du sprint 5 EPIC14 (chapitre 12).
- **Honeycomb BubbleUp** fait émerger les distributions de valeurs de caractéristiques qui distinguent une population de requêtes lentes d’une population rapide — un outil efficace de bas en haut, mais qui suppose le modèle de données à haute cardinalité spécifique de Honeycomb.
- **Komodor** produit des chronologies narratives d’incidents pour des charges Kubernetes. Les archétypes d’exemplaires et la collecte en profondeur des traces de la plateforme CIRES (planifiée via `get_trace`) couvrent un espace conceptuel similaire.

Le précédent publié le plus proche de la forme générale de la plateforme est le corpus de travaux autour de l’usage de sortie structurée par LLM dans l’ingénierie du support (« LLM comme copilote de RCA » plutôt que « LLM comme répondant autonome »). Les contributions originales de ce projet sont (a) l’invariant MCP-seul traité comme règle à l’échelle du projet, (b) la bibliothèque d’exemples des quatorze archétypes canoniques de RCA injectée comme échafaudage de *prompt* et (c) le couple validateur de forme de prose / bridage de confiance qui constituent ensemble le pare-feu anti-hallucination.

CHAPITRE 3

Découverte et recueil des besoins (sprint 0)

3.1 Démarche

Le projet a conduit un sprint 0 délibéré entre février et les premiers jours de mars 2026, avant qu’aucune ligne de code d’infrastructure ne soit écrite. Deux pistes parallèles ont couru pendant toute la durée de ce sprint :

1. **Étalonnage des solutions de supervision.** Un examen des principales piles d’observabilité *open source*, de leurs matrices de fonctionnalités, de leurs modèles de déploiement et de leur empreinte d’intégration avec la pile JVM / Spring Boot déjà en place chez CIREs. Les alternatives SaaS (Datadog, New Relic, Honeycomb) ont été chiffrées et écartées pour cet engagement, mêlant raisons budgétaires et raisons de souveraineté des données.
2. **Recherche de terrain.** Conversations structurées avec les ingénieurs et les opérateurs les plus proches du système de production, centrées sur trois questions : ce qui les réveille, comment ils découvrent d’abord qu’une anomalie est en cours et quelle est leur première étape d’investigation lorsque une alerte arrive.

Un troisième flux, mené en parallèle, a consisté en une lecture guidée du *SRE Book* et du *SRE Workbook* de Google, avec une traduction explicite du vocabulaire SRE dans le contexte opérationnel du projet.

3.2 Constats issus des entretiens

Le constat le plus marquant issu de la recherche de terrain est revenu dans presque toutes les conversations. À la question « comment les problèmes étaient-ils d’abord découverts ? », la réponse modale a été une variante de :

Constat récurrent des entretiens

« Nous découvrons que quelque chose est cassé parce qu’un client nous le dit, ou parce que quelqu’un s’en aperçoit après coup en relisant les journaux. »

La détection accusait un retard de plusieurs heures à plusieurs jours sur la défaillance réelle. La posture réactive de l’équipe était structurelle, et non une question d’effort : rien, dans le système, n’attirait l’attention sur un problème de sa propre initiative. Ce constat a cristallisé l’énoncé du problème donné au §1 et a façonné toute l’architecture qui a suivi.

Un second constat, moins central mais important, a été qu’*même lorsque* des alertes étaient en place (par exemple sur l’usage du disque de certains serveurs), les opérateurs devaient régulièrement mener un travail d’investigation non trivial pour comprendre ce que l’alerte signifiait dans le contexte courant. L’insistance du projet sur des alertes *pré-investiguées* — arrivant avec une RCA candidate, des preuves et une remédiation — est la conséquence directe de ce second constat.

3.3 Exploration de l'espace des solutions

Plusieurs architectures candidates ont été imaginées et mises à l'épreuve avant de retenir la conception finale à deux couches :

1. **Alerte traditionnelle par règles uniquement.** Rejetée — elle existe déjà conceptuellement, ne traite que la moitié *détection* du problème, et laisse la moitié *investigation* à un humain qui doit lire les tableaux de bord au bon moment.
2. **Couche de résumé par IA au-dessus des alertes brutes.** Plus proche : aborde partiellement la moitié *investigation*, en donnant à l'opérateur un résumé narratif à côté de l'alerte. Risquée car le résumé est purement descriptif (« le CPU est élevé ») plutôt que diagnostique (« le CPU est élevé parce que le *cron job* tourne et rate son verrou ») ; cela ne réduit pas réellement la charge d'investigation de l'opérateur.
3. **Couche de tri par IA complète qui transforme les alertes en RCA structurées avec verdicts et suggestions de remédiation.** Retenue. Traite les deux moitiés : la couche d'observabilité détecte, la couche de tri pré-investigue. Risques explicitement consignés : hallucination, sortie de surface uniquement, dépendance à la disponibilité d'un LLM auto-hébergé, budget de latence de l'appel LLM face aux attentes de l'opérateur.

3.4 Exigences fonctionnelles

1. **FR-1** La plateforme doit ingérer les télémétries métriques, journaux et traces de la pile web cible sans modification du code source des services applicatifs. L'instrumentation doit être *externe* : agents montés au démarrage, *plug-ins* configurés à la couche passerelle.
2. **FR-2** La plateforme doit lever des alertes sur les symptômes observables par l'utilisateur (latence, taux d'erreurs, trafic) et sur un ensemble de signaux d'appoint en forme de causes (CPU, mémoire, disque, cible en panne, anomalies de journal).
3. **FR-3** Pour chaque alerte qui n'est pas supprimée par les couches de déduplication ou de barrière de récurrence, la plateforme doit produire une RCA structurée contenant : verdict (escalade / rejet / non concluant), prose de la cause racine, actions suggérées ou étapes de diagnostic en lecture seule, liste des preuves citées, liste des alertes corrélées dans la fenêtre environnante et une valeur de confiance.
4. **FR-4** La RCA doit être persistée dans un magasin d'historique durable, exposée via un tableau de bord et envoyée comme courriel d'escalade lorsque le verdict l'indique.
5. **FR-5** Les corrections de l'opérateur sur les RCA (« en fait, rejeter » / « en fait, escalader ») doivent être capturées et utilisées comme signal de routage pour les déclenchements ultérieurs.
6. **FR-6** Toutes les sources de données consommées par le LLM doivent être accédées via un pont MCP (l'invariant MCP-seul).

3.5 Exigences non fonctionnelles

1. **NFR-1 Reproductibilité.** La pile complète doit être reproductible depuis une infrastructure vierge via un unique *playbook* Ansible (sprint 1) ou un couple Terraform + Ansible (sprint 2).

2. **NFR-2 *Open source* d’abord.** Aucune dépendance SaaS hébergée sur le chemin critique. Un LLM peut être un fichier de modèle tiers, mais doit pouvoir s’exécuter localement sur du matériel grand public.
3. **NFR-3 Budget de latence.** Délai entre l’arrivée du webhook et la persistance de la RCA : cible < 2 min à chaud, < 3 min à froid sur le *runner* portable. Ce budget est tenu (typiquement 25–68 s) et documenté au chapitre 9.
4. **NFR-4 Coût.** Cible totale de coût mensuel d’exploitation : sous 100 \$/mois, hors ressources éligibles au *Free Tier* AWS. Ce budget motive la décision d’abandonner la migration GPU du sprint 4 (§11.5).
5. **NFR-5 Auditabilité.** Chaque modification de la plateforme doit être revisable dans le gestionnaire de versions ; les décisions durables de conception doivent être consignées dans un journal des décisions ; les audits quotidiens statiques et en condition réelle doivent produire une trace écrite.

CHAPITRE 4

Architecture du système

4.1 Vue d'ensemble en couches

La plateforme est structurée en trois couches, illustrées à la figure 4.1.

Couche 3 — tri par IA Service FastAPI, qwen2.5 :7b sur Ollama, tableau de bord, escalade SMTP
Couche 2 — pont MCP prometheus-mcp, loki-mcp, jaeger-mcp, rca-history-mcp, deploy-events-mcp
Couche 1 — fondation d'observabilité Prometheus, Loki, Jaeger, Grafana, OTel Collector, node-exporter, cAdvisor, <i>poller</i> Drain3
Couche 0 — sources de télémétrie Spring Boot (agent OTel), Kong (<i>plug-in</i> OTel), MySQL, pods k3s, métriques d'hôte

Figure 4.1. Architecture en couches. Chaque couche ne consomme que la couche immédiatement inférieure ; l'invariant MCP-seul est la règle qui formalise ce principe pour la couche IA.

L'agencement est délibérément conservateur. Les alertes conventionnelles remontent de la couche 0 à travers Prometheus et Grafana de la manière habituelle ; la couche IA est un consommateur supplémentaire qui ne se substitue pas à l'acheminement des alertes existant. Si la couche IA est mise hors service, la plateforme se dégrade gracieusement vers une configuration standard d'alerte Grafana ; l'inverse n'est pas vrai.

Les figures 4.2 et 4.3 rendent la même architecture sous forme de graphe de composants, séparée selon les deux phases livrables du projet. La phase 1 est la fondation d'observabilité classique ; la phase 2 est la couche de tri par IA construite par-dessus. Les figures utilisent un même vocabulaire de couleurs entre les deux panneaux afin que l'œil puisse suivre un signal de l'émission d'un service jusqu'à un courriel d'escalade ; la figure 4.4 présente la vue combinée à une échelle légèrement réduite pour la mise en correspondance.

4.2 Topologie d'exécution

Après la migration du sprint 2, la topologie d'exécution de la plateforme se répartit sur trois classes d'hôtes :

1. **Hôte de supervision** (`observability-rca-monitoring` en `us-east-1`, EC2 `t3.large`). Exécute la pile d'observabilité comme un déploiement Docker Compose à sept conteneurs : Prometheus, Loki, Jaeger, Grafana, OTel Collector, cAdvisor, node-exporter.
2. **Hôte applicatif** (`observability-rca-k3s` en `us-east-1`, EC2 `t3.xlarge`). Exécute la charge applicative (Spring Boot, Kong, MySQL, *front-end*) sur un cluster k3s à un seul nœud. Un DaemonSet node-exporter fournit les métriques d'hôte du côté k3s.
3. **Hôte IA** (le portable de l'ingénieure, WSL2 + Docker Desktop, NVIDIA GTX 1060 6 Go). Exécute le service de tri, les cinq serveurs MCP et Ollama avec `qwen2.5:7b-instruct` chargé. Le portable est joignable sur un *tailnet* Tailscale et adressé par MagicDNS (`adolin-wsl`).

Les trois hôtes partagent un *tailnet* Tailscale qui contient également le nœud contrôleur de l'opérateur. WireGuard assure le transport ; les adresses IP publiques des instances AWS ne sont pas exposées pour le trafic de service.

4.3 Flux de données : de l'alerte à la RCA

Le flux de bout en bout d'une alerte à travers la plateforme se déroule en dix étapes, résumées à la figure 4.5.

Ce pipeline est décrit étape par étape aux chapitres 6 et 7 ; les mécanismes de validation et de bridage de la confiance introduits aux étapes 6b et 6d sont au cœur du thème du pare-feu anti-hallucination du projet et sont traités en détail au chapitre 7. La figure 4.6 rend le même pipeline sous forme d'organigramme vertical avec les étapes de déduplication, de suppression, de barrière de récurrence, du LLM, du validateur et de notification disposées de haut en bas ; les deux sorties latérales (supprimée précocement, écartée par le validateur) sont explicites dans cette vue.

4.4 L'invariant d'accès aux données via MCP uniquement

Invariant MCP-seul

Chaque donnée que voit le LLM dans son *prompt* doit transiter par un serveur MCP. Les appels directs `httpx` ou `requests` vers Prometheus, les lectures directes `aiosqlite` de la base d'historique RCA, et les imports directs en mémoire `from app.exemplars import ...` sont tous interdits dans les chemins de code adjacents au LLM.

L'invariant sert deux objectifs. Premièrement, il contraint la provenance : chaque fait montré au LLM peut être tracé jusqu'à un appel d'outil MCP spécifique, avec horodatages et codes de réponse ; il n'y a aucune ambiguïté sur « où le modèle a-t-il vu ceci ? » Deuxièmement, il élimine une classe de causes racines d'hallucination ; un import Python en mémoire peut réussir silencieusement et faire apparaître des données périmées ou partielles, alors qu'un appel MCP dispose d'une enveloppe d'erreur explicite et est observable dans les métriques de la couche pont.

Trois violations de l'invariant ont été détectées lors de l'incident `0b215ef3` du 2026-04-29 et constituent le périmètre des user stories de l'Épopée 4 (pare-feu anti-hallucination) S3-HF-04 / S3-HF-05 / S3-HF-06. Voir les chapitres 8 et 7 pour le diagnostic et le plan de correction.

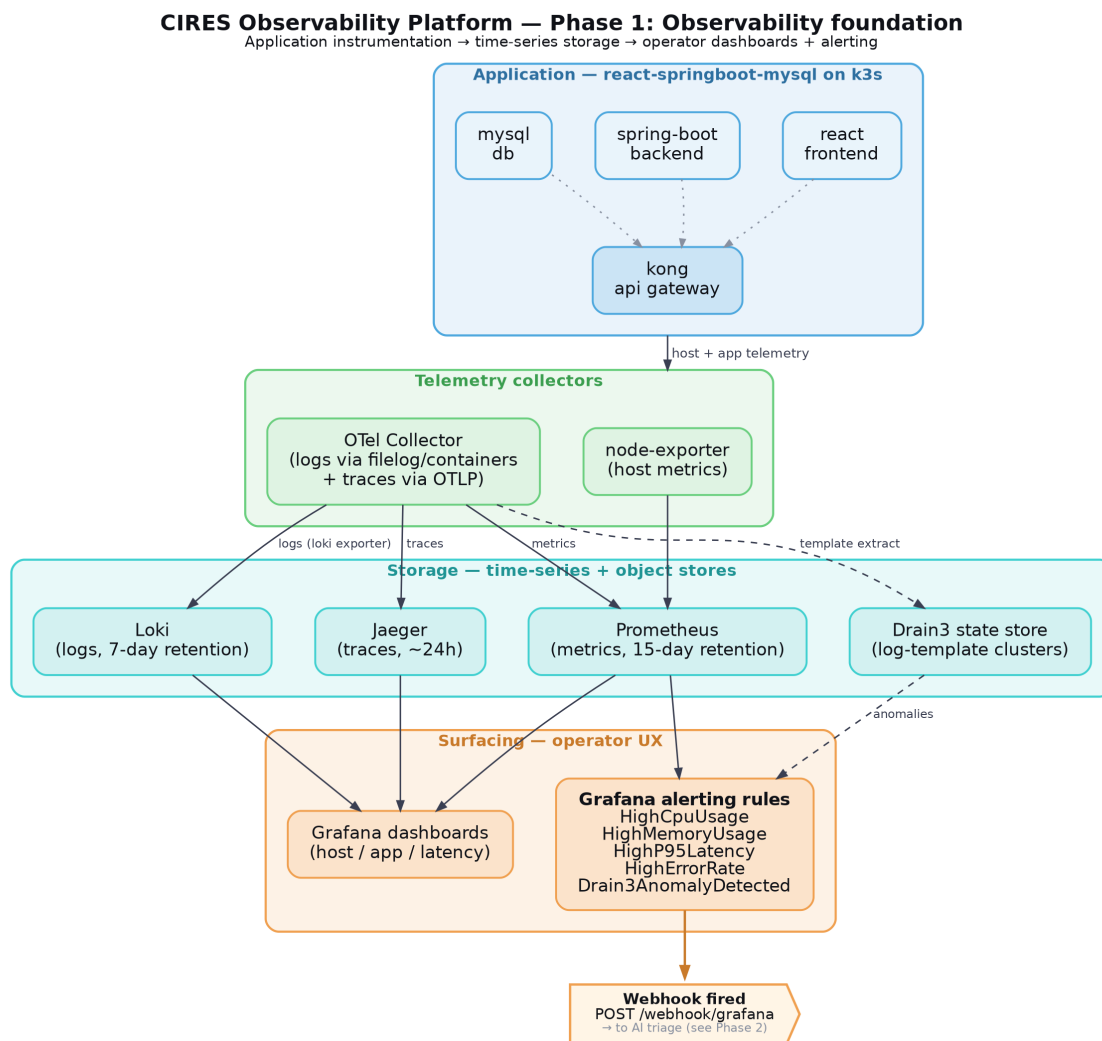


Figure 4.2. Phase 1 — fondation d’observabilité. L’instrumentation applicative (Spring Boot, Kong, MySQL, k3s) alimente les collecteurs (scrape Prometheus, OTel Collector avec son récepteur `filelog/containers` pour les journaux et son récepteur `OTLP` pour les traces) et le stockage (Prometheus TSDB, Loki, Jaeger). Grafana expose les données et achemine les alertes via le point de contact `webhook` de l’alerting unifié.

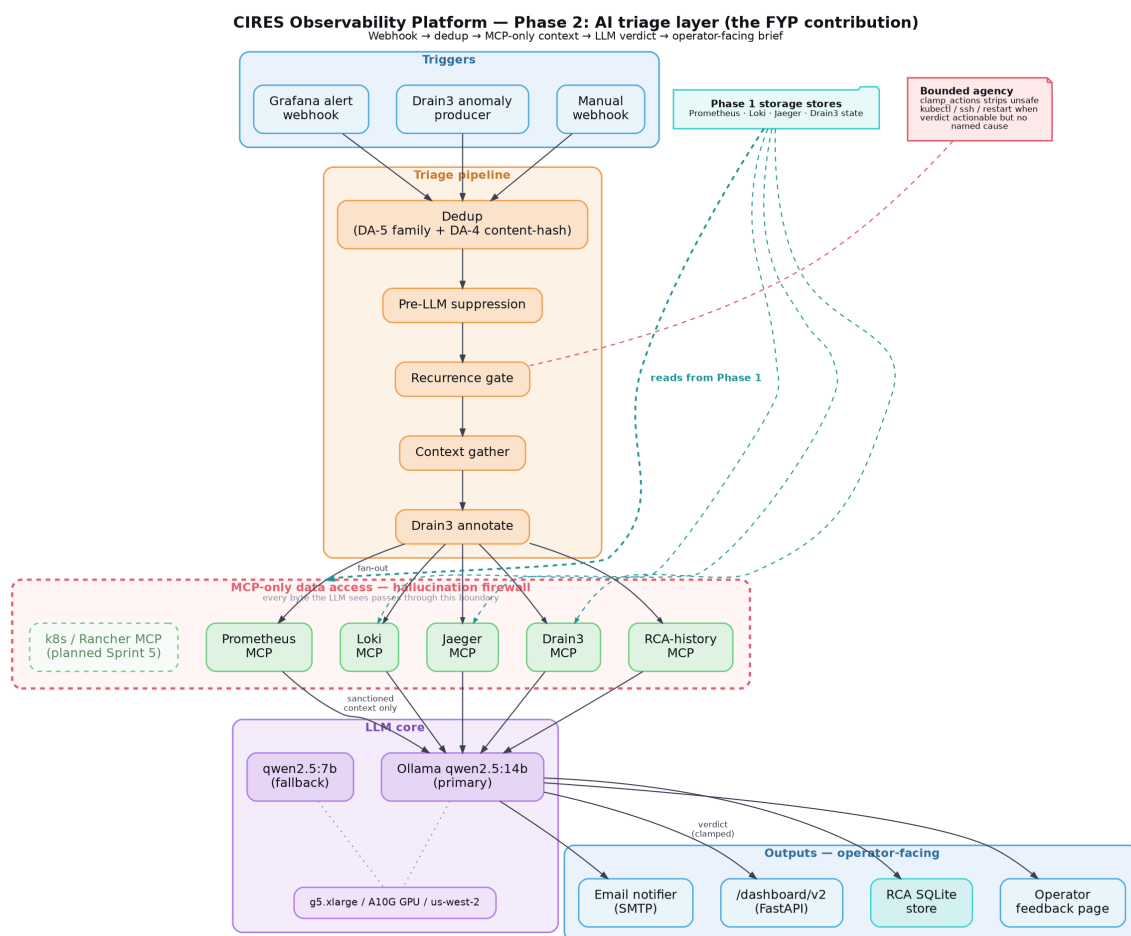


Figure 4.3. Phase 2 — couche de tri par IA. Les récepteurs de webhooks (Grafana, Drain3) alimentent le pipeline en dix étapes ; la collecte de contexte interroge en parallèle les cinq ponts MCP ; le LLM est invoqué une seule fois sur le bloc de preuves pré-collecté ; les validateurs, le bridage de confiance et la reprise à action contenue ferment la boucle avant que la RCA ne soit persistée, affichée sur le tableau de bord et envoyée comme courriel d’escalade.

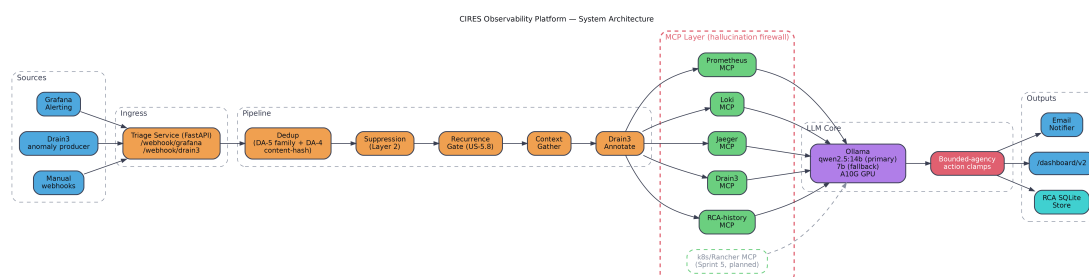


Figure 4.4. Vue d’ensemble complète de la plateforme (phase 1 + phase 2) à échelle réduite, conservée pour la mise en correspondance avec les panneaux séparés ci-dessus. Une variante *de style designer* générée en matplotlib ([architecture-phase1-observability-v2.png](#)) est conservée dans le dépôt comme alternative stylistique.

Étape	Ce qui se passe	Module
1	Le webhook arrive depuis Grafana ou le <i>poller</i> Drain3	main.py
2	Déduplication et suppression à deux niveaux	dedup.py
3	Collecte de contexte (Prom, Loki, Jaeger, Drain3, historique RCA)	context.py
4	Interprétation des métriques numériques	metric_interpreter.py
5	Injection d'un exemplaire (1 sur 14 archétypes)	exemplars/
6	Appel LLM (sortie structurée, qwen2.5 :7b)	llm_client.py
6b	Validation de la réponse (surface, menu d'hypothèses, liste noire)	response_validator.py
6c	Reprise à action contenue (un appel MCP max)	bounded_agency.py
6d	Bridage de la confiance (retrait des actions si non fiable)	pipeline.py
7	Repli sur modèles d'actions (si la liste d'actions de la RCA est vide)	action_templates.py
8	Persistance dans <code>rca_history</code> , envoi du courriel	rca_store.py, notifier.py

Figure 4.5. Le pipeline de tri en dix étapes. Les étapes 6/6b/6c/6d forment ensemble le pare-feu anti-hallucination.

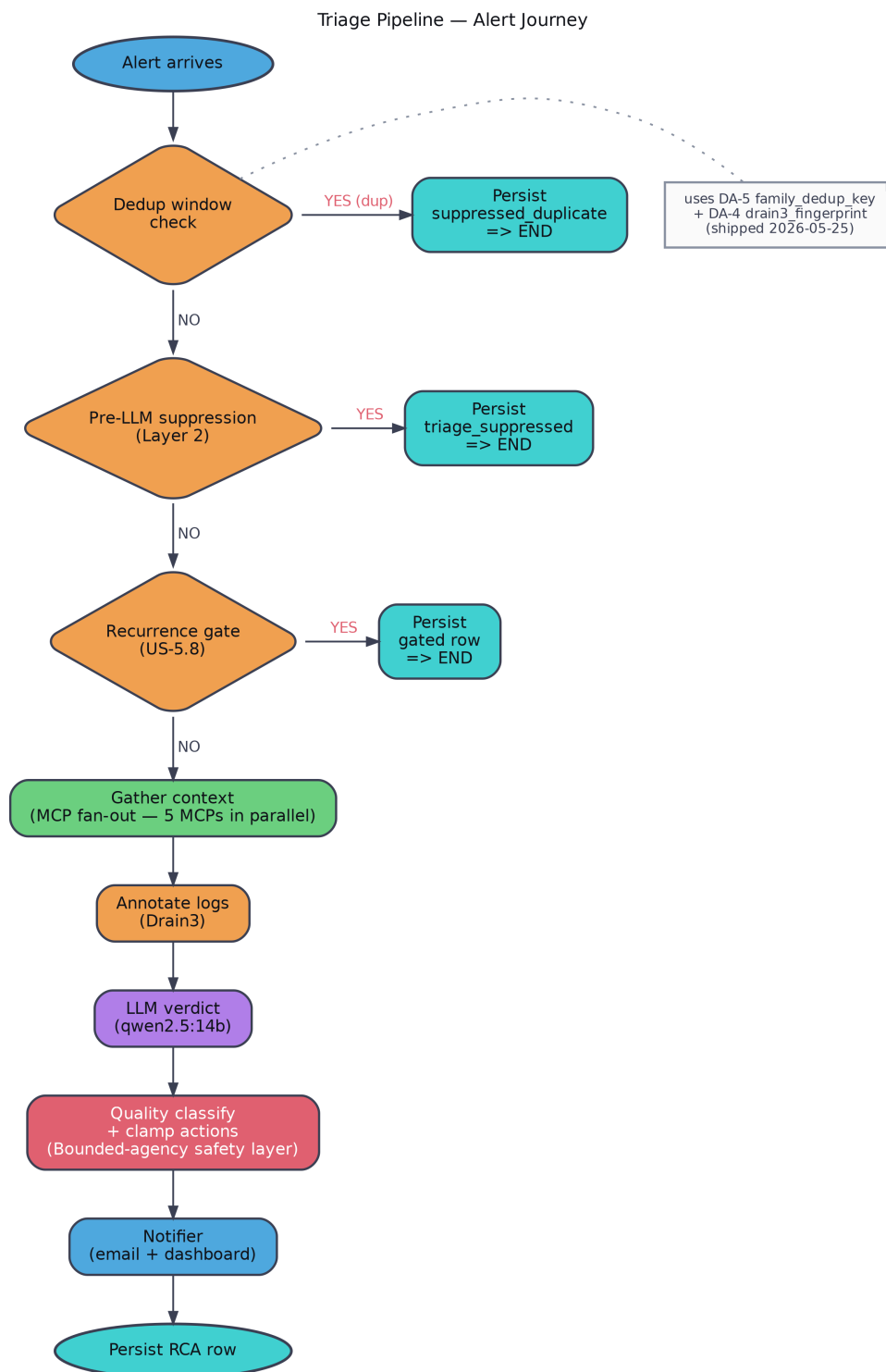


Figure 4.6. Le pipeline de tri en organigramme vertical. Une alerte descend depuis le récepteur de webhooks à travers les étapes de déduplication, de suppression et de barrière de récurrence, vers la collecte de contexte, l'appel LLM, la pile de validateurs et les étapes de persistance et de notification. Les deux sorties latérales précoces (supprimée avant LLM, écartée par le validateur) sont la défense de la conception contre les appels LLM gaspillés et les sorties opérationnellement non fiables.

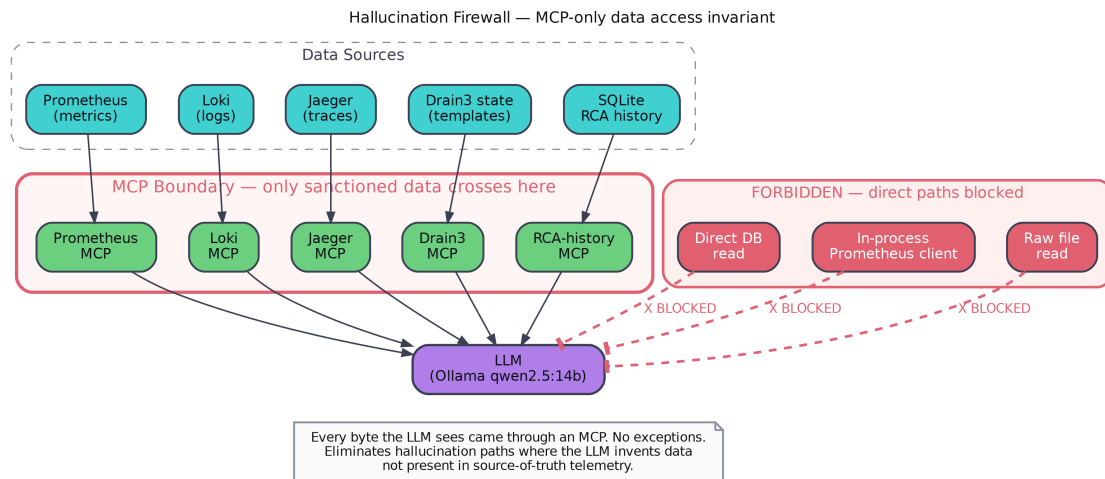


Figure 4.7. L’invariant d’accès aux données via MCP uniquement représenté comme un diagramme de pare-feu. Le LLM ne voit que ce qui arrive par l’un des cinq ponts MCP ; les chemins en pointillés montrent les trois accès directs (en mémoire ou en HTTP direct) que l’incident `0b215ef3` a exposés et que l’épopée anti-hallucination du sprint 3 a refermés. Le lint d’intégration continué livré comme S3-HF-08 est la couche d’application mécanique.

CHAPITRE 5

Sprint 1 — Fondation d’observabilité

5.1 Objectif du sprint et topologie

Le sprint 1 s’est déroulé du 2026-03-10 au 2026-03-30. L’objectif était d’exécuter la première moitié de l’hypothèse du sprint 0 : déployer une plateforme d’observabilité de bout en bout pour la pile web représentative à l’aide d’outils *open source*, déployable par quiconque dispose du *playbook*. Aucune couche IA n’est présente à ce stade.

La topologie du sprint 1 reposait sur trois machines virtuelles sur site : `monitoring-vm` (.10), `application-vm` (.11), `network-vm` (.12), sur le sous-réseau 192.168.127.0/24. Les trois étaient des hôtes Ubuntu LTS standards avec Docker installé ; l’intégralité de la pile était déployée sous forme de services Docker Compose pilotés par Ansible.

5.2 Disposition des rôles Ansible

Un rôle par service a été établi, chacun idempotent et paramétré, avec un template Docker Compose, une génération de fichier d’environnement et une tâche de *health-check* :

```
roles/prometheus, roles/loki, roles/jaeger, roles/grafana, roles/otel_collector, roles/kong,
roles/spring_boot, roles/cadvisor, roles/node_exporter, roles/mysql.
```

Un unique *playbook* (`playbooks/site.yml`) amenait un ensemble de VM fraîchement provisionnées à un état pleinement observable en environ quinze minutes. Ce *playbook* fait partie des preuves de reproductibilité du projet et est documenté dans `monitoring-docs/reproducibility-guide.html` et `build-guide.html`.

5.3 Télémétrie à trois piliers

- **Métriques** via Prometheus, en scrapant `/actuator/prometheus` de Spring Boot, l’endpoint de statistiques de Kong, node-exporter sur chaque VM, et cAdvisor sur la VM applicative.
- **Journaux** via Loki, avec Promtail qui suit les journaux des conteneurs Docker sur chaque VM et les expédie avec un petit ensemble d’étiquettes bien contrôlées (`service_name`, `instance`, `container`).
- **Traces** via l’OpenTelemetry Collector, recevant les *spans* de l’agent Java OTel injecté dans le processus Spring Boot au démarrage et du *plug-in* OTel installé sur la passerelle Kong. Les identifiants de trace se propageaient de bout en bout, de sorte qu’une requête unique apparaissait comme une seule trace, de la passerelle à l’*upstream*.

5.4 Grafana et alerting initial

Grafana a été provisionné via des templates Ansible et livré avec quatre tableaux de bord canoniques (vue d’ensemble des services, vue d’ensemble des alertes, explorateur de journaux, explorateur de traces) installés sous `/var/lib/grafana/dashboards`. L’alerting initial utilisait

Alertmanager, avec des règles de seuil métrique sur CPU, mémoire, disque et état *target-down*, acheminées via un point de contact SMTP Gmail. Alertmanager a été retiré au sprint 2 (décision D6) au profit de l’alerting unifié Grafana avec un unique point de contact **triage-webhook** alimentant le service de tri par IA.

5.5 Résultat du sprint 1

Une pile d’observabilité reproductible à trois piliers pour une application web représentative, déployable depuis des VM vierges en environ quinze minutes via une unique commande Ansible. Aucune modification du code source applicatif requise ; l’instrumentation est entièrement externe (agent OTel, *plug-in* OTel, endpoints de scrape, Promtail). C’est la fondation sur laquelle s’appuient tous les sprints suivants.

Les éléments reportés à la clôture du sprint étaient : la topologie à trois VM (remplacée au sprint 2 par AWS EC2 + k3s) ; Alertmanager (retiré au sprint 2 par D6) ; le point de contact SMTP Gmail (remplacé par l’envoi SMTP du service de tri lui-même) ; et l’espace d’adressage IP sur site (remplacé par un *tailnet* Tailscale).

Promtail a été retiré le 2026-03-30 et remplacé par l’OTel Collector. Le récepteur `filelog/containers` du collecteur capte les journaux des conteneurs Docker depuis `/var/lib/docker/containers/**/*-json.log` et l’exportateur natif `loki` les pousse vers `loki:3100/loki/api/v1/push`. Cela unifie l’ingestion logs + traces (auparavant scindée entre Promtail pour les journaux et l’OTel Collector pour les traces) et élimine un démon par hôte.

CHAPITRE 6

Sprint 2 — Migration AWS et MVP RCA par IA

6.1 Objectif du sprint

Le sprint 2 s'est déroulé du 2026-03-31 au 2026-04-23, avec un prolongement (Épopée 5) jusqu'au 2026-04-29. L'objectif était de faire passer la plateforme des machines virtuelles sur site vers AWS (EC2 provisionnée par Terraform), de migrer la charge applicative vers k3s, et d'ajouter une couche IA qui produit des analyses structurées des causes racines à partir des alertes entrantes. La fondation sur site héritée du sprint 1 reste utile comme cible de reproductibilité ; le déploiement de forme production vit sur AWS.

L'échéance du sprint a glissé deux fois (du 9 avril au 23 avril), mouvement capturé par les ratures dans le *tracker* de progression du projet. Atterrissage final : 2026-04-23.

6.2 Épopée 1 — Infrastructure AWS

Une configuration Terraform dans le dépôt `provisioning-monitoring-infra` gère l'intégralité du patrimoine AWS : un VPC avec sous-réseaux publics et privés, deux instances EC2 (`t3.large` pour la supervision, `t3.xlarge` pour k3s), une instance RDS MySQL `db.t3.micro`, des *Elastic IPs* sur les hôtes pérennes, des groupes de sécurité paramétrés par le CIDR Tailscale plus un CIDR d'IU publique pour l'accès aux interfaces Grafana / Prometheus / Loki / Jaeger, des rôles IAM par instance, un *bucket* S3 pour le stockage froid de Loki et un second pour l'état Terraform. L'AMI Ubuntu a été figée sur un identifiant d'image spécifique pour supprimer la dérive silencieuse entre exécutions.

6.3 Épopée 2 — MVP du système RCA par IA

Cette épopée introduit la plus grande nouvelle surface du projet. Elle comporte :

- **Cinq serveurs MCP** (`monitoring-mcp-servers/`). Chacun est un petit processus Python qui expose trois à quatre outils sur un transport JSON-RPC, avec un endpoint `/metrics` de style Prometheus pour l'observabilité du pont lui-même.
- **Service de tri FastAPI** (`monitoring-triage-service/`). Récepteurs de webhooks pour Grafana et Drain3, pipeline asynchrone en dix étapes, appels LLM à sortie structurée via Ollama, déduplication et suppression, tableau de bord, et escalade SMTP.
- **Corrélation transversale aux piliers**. Le module `context.py` interroge trois MCP en parallèle avant l'appel LLM, construisant un unique bloc de preuves pré-collecté que le LLM consomme. Cela tient le modèle à l'écart de la décision « quelles requêtes lancer » ; c'est la plateforme qui décide.
- **Regroupement de modèles de journaux Drain3**. Côté serveur sur Loki, faisant émerger les modèles nouveaux comme quatrième signal aux côtés des métriques, journaux et traces.

- **LLM local.** `qwen2.5:7b-instruct` via Ollama, d’abord sur une EC2 `g4dn.xlarge` (décommissionnée), puis déplacé sur le GTX 1060 du portable le 2026-04-23 (toujours le *runner* de production aujourd’hui).

6.4 Épopée 3 — Migration Kubernetes

La charge applicative a été déplacée depuis le Docker Compose de `application-vm` vers un cluster k3s à un seul nœud sur `observability-rca-k3s`. Des *Helm charts* ont été rédigés pour le *back-end* Spring Boot, Kong et le *front-end*. Le rôle Ansible Galaxy `xanmanning.k3s` provisionne k3s comme unité `systemd`; un DaemonSet `node-exporter` fournit les métriques d’hôte du côté k3s.

La pile de supervision est restée sur Docker Compose. La décision D11 en consigne le motif : la VM de supervision est opérationnellement stable, déployable depuis Ansible, et ne gagne rien de structurel à migrer vers Kubernetes; le coût de la migration (*Helm charts* pour neuf services, gestion des *persistent volume claims*, courbe d’apprentissage opérationnelle propre à Kubernetes) excédait le bénéfice.

6.5 Épopée 4 — Renforcement du tri par IA

Cette épopée fait passer la couche de tri par IA d’un « MVP qui répond aux webhooks » à un « service auquel les opérateurs peuvent se fier » : une couche de déduplication à deux niveaux (empreinte en mémoire plus une suppression pré-LLM fondée sur l’historique — décision D4), l’application du schéma de sortie structurée à la couche d’appel LLM (D12), une enveloppe d’erreur sur chaque réponse MCP, des *timeouts* de pipeline, le tableau de bord opérateur, le notificateur courriel à carte bordée selon la confiance, et une première version du repli sur modèles d’actions.

6.6 Épopée 5 — User stories à coloration UEBA (post-sprint)

Cinq user stories d’inspiration UEBA (*User and Entity Behavior Analytics*) ont couru au-delà de la clôture du sprint comme flux de travail cohérent en cours et ont été *in fine* intégrées au sprint 3 sous l’épopée « clôture de la qualité RCA v2 ». L’effort net est d’environ quarante-neuf points d’histoire livrés après la clôture du sprint. Ces user stories sont résumées au tableau 6.1.

Story	Ce qu’elle a ajouté	Résultat
US-5.1 Phase A	Drain3 par service — dictionnaire <code>service</code> → <code>TemplateMiner</code> avec état sur disque par service.	Livrée le 2026-04-28
US-5.1 Phase B	Lignes de base d’entités — quantiles glissants sur 7 jours par couple service-métrique + auxiliaire de <i>claim</i> σ .	Livrée le 2026-04-28; vérification clôturée par S3-HF-04
US-5.3	Retour d’information en boucle fermée override/confirmation + métriques <code>triage_precision</code> / <code>triage_recall</code> en évaluation paresseuse.	Livrée le 2026-04-28
US-5.4	Seuils adaptatifs sur les trois règles les plus flottantes (même-heure-il-y-a-7-jours + ligne de base 3σ).	Livrée le 2026-04-27
US-5.8	Barrières de récurrence (pre-LLM + post-LLM, opt-in pour Medium-CPU/Mem).	Livrée le 2026-04-28

Table 6.1. User stories de l’Épopée 5 reportées au-delà de la clôture du sprint 2.

6.7 Résultat du sprint 2

La transition depuis « nous avons de l’observabilité » (sprint 1) vers « la plateforme d’observabilité produit de bout en bout des RCA structurées et validées avec bouclage du retour d’information de l’opérateur » (sprint 2 + Épopée 5). Le pipeline qui tourne aujourd’hui en production est l’architecture du sprint 2 surmontée des mécanismes de qualité et de sûreté du sprint 3. La suite de tests a atteint 178/178 à la clôture du sprint sur le service de tri.

CHAPITRE 7

Sprint 3 — Qualité, retour d’information et pare-feu anti-hallucination

7.1 Forme du sprint

Le sprint 3 court du 2026-04-23 au 2026-05-19 (étendu depuis la clôture initialement prévue le 05-08). Quatre épopées :

1. **Épopée 1 — clôture qualité RCA v2 (49 SP).** Reprise des travaux de l’Épopée 5 livrés entre le 2026-04-23 et le 2026-04-29 sous le registre d’épopée du sprint 3. Douze user stories sur treize terminées ; une story de nettoyage reste à faire.
2. **Épopée 2 — boucle de retour d’information de l’opérateur (7 SP).** Extension du schéma de la table de retour US-5.3 existante ; nouveaux endpoints ; quatre nouveaux outils MCP sur `rca_history_mcp` ; formulaire de notation dans le courriel. Non démarrée à l’heure d’écriture.
3. **Épopée 3 — cycle de vie des lignes de base Drain3 (6 SP).** Câblage de la tuyauterie S3 dormante dans `drain_analyzer.py` ; instantané hebdomadaire via APScheduler ; restauration au démarrage ; trois endpoints opérateur ; mise à jour IAM. Non démarrée à l’heure d’écriture.
4. **Épopée 4 — pare-feu anti-hallucination + profondeur des traces (15 SP, clôturée 2026-05-19).** Ajoutée le 2026-04-29 après l’incident 0b215ef3. Deux thèmes : (a) refermer les trois contournements de l’invariant MCP-seul sur le chemin de collecte du LLM ; (b) câbler l’outil MCP `get_trace(trace_id)` qui existait mais n’était jamais invoqué. Cinq SP livrés le jour même de l’incident ; les dix SP restants (S3-HF-04 à S3-HF-09) livrés en une session ciblée à la clôture du sprint, le 2026-05-19.

Le sprint comporte une fenêtre d’investigation délibérée de vingt jours (du 2026-04-30 au 2026-05-19) en plein milieu de son calendrier ; le travail mené pendant cette fenêtre est décrit au §7.10.

7.2 La grille de qualité RCA à 4 axes

Pour rendre « une bonne RCA » tractable comme cible d’ingénierie, le projet définit une grille à quatre axes utilisée pour noter chaque RCA produite, à la fois dans l’automatisation des runs de chaos et dans l’audit quotidien en condition réelle :

1. **Cause nommée** — la RCA nomme une cause racine et non une simple reformulation de l’alerte (par exemple « épuisement du pool de connexions sur `spring-boot` » plutôt que « l’alerte indique une latence élevée »).
2. **Preuves citées** — la cause est étayée par des valeurs métriques, lignes de journal ou attributs de trace spécifiques tirés du contexte pré-collecté.

3. **Actionnable** — la RCA porte soit des actions suggérées opérationnellement valides, soit, dans le cas supprimé, des étapes de diagnostic en lecture seule valides.
4. **Forme de la prose** — la RCA commence par la cause plutôt que par une expression PromQL, une observation ou une ouverture « sur la base du contexte » (la philosophie énoncée par la décision D19).

Chaque axe est noté sur une échelle de 0 à 1 ; la moyenne non pondérée donne la qualité RCA globale. La grille est implémentée dans `monitoring-project/scripts/chaos/lib/rca_scorer.py` et utilisée à la fois par le banc d'injection de chaos et par l'audit quotidien en condition réelle.

7.3 La bibliothèque d'exemples (D17)

Décision D17 — injection d'exemplaires

Le LLM se voit montrer un parmi quatorze archétypes canoniques de RCA comme cible structurelle avant de voir le bloc de preuves. Chaque archétype contient un motif de *alertname*, un filtre service-et-type-de-déploiement, un pilier de signal (métrique / journal / trace / drain3 / mixte), une classe de sévérité et un court exemple de RCA dans la forme de prose préférée du projet.

Les quatorze archétypes sont : `oom-loop`, `upstream-latency-attribution`, `service-self-p95-saturation`, `critical-resource-saturation`, `otel-collector-degradation`, `synthetic-blip-dismiss`, `cascade-incident-disk-loki`, `drain3-novelty-post-deploy`, `bounded-agency-resolves-inconclusive`, `adaptive-threshold-noop`, `closed-loop-feedback-override`, `crashloop-bad-config`, `network-firewall-tls-cert-expiry-pre-failure`. Un repli par défaut `generic-sre-shape` est utilisé quand aucun ne correspond.

La sélection passe par `exemplars.find_for_alert(alertname, service, deployment_type, signal, severity)` qui note les quatorze sur une correspondance pondérée (regex de *alertname* 0,10 ; services 0,40 ; type de déploiement 0,30 ; signal 0,20 ; sévérité 0,10) et renvoie le plus haut. L'exemplaire correspondant est rendu comme section **## Reference exemplar avant le ## Pre-Gathered Context** dans le *prompt* — la position est porteuse de sens et a été confirmée pendant la fenêtre d'investigation (§7.10) comme surperformant la position post-contexte sur la grille à 4 axes.

7.4 Seuils adaptatifs (D18)

Les seuils numériques statiques (`p95 > 1000ms`, `cpu > 80%`) génèrent des flots de faux positifs en heures creuses et ratent les anomalies réelles en heures de pointe. Le trafic réel n'est pas stationnaire ; les règles ne doivent pas faire semblant qu'il l'est. Trois règles Grafana (`HighP95Latency`, `HighKongP95Latency`, `MediumCpuUsage`) ont été converties à une ligne de base même-heure-il-y-a-7-jours plus une enveloppe à 3σ . Les seuils s'ajustent sur une fenêtre glissante hebdomadaire et l'alerte ne se déclenche que lorsque la valeur courante sort de l'enveloppe historique pour l'heure correspondante.

7.5 Philosophie de la prose RCA (D19)

La décision D19 capture la position selon laquelle *la télémétrie est le moyen, pas la fin* : un paragraphe RCA qui commence par une expression PromQL ou une valeur métrique observée se

lit comme la requête de l'alerte, pas comme une investigation. Trois surfaces complémentaires ont été mises à jour pour imposer une forme de prose orientée cause :

- Une nouvelle règle J et la règle A du `SYSTEM_PROMPT` instruisent explicitement le modèle de commencer par la cause ; la version précédente de la règle A demandait au modèle de commencer par l'expression `PromQL` et la valeur observée, surface d'entraînement involontaire pour des sorties de surface uniquement.
- Les onze paragraphes `rca` d'exemplaires ont été réécrits pour épouser la forme cause-en-tête.
- Une nouvelle règle de validateur scanne la RCA produite contre un ensemble de regex `_SURFACE_ONLY_LEDE_PATTERNS`. Les correspondances déclenchent une reprise sous le chemin à action contenue.

7.6 Retour d'information en boucle fermée (D20)

Les corrections d'opérateur sur les RCA étaient auparavant silencieuses : le LLM produisait un verdict ; l'opérateur était d'accord ou non ; le système n'apprenait jamais. Sans ce signal, toute modification de *prompt*, d'exemplaire ou de seuil était non mesurable.

La décision D20 introduit une couche en boucle fermée. Une table `feedback` consigne le verdict réel de l'opérateur ($\pm$ un texte libre `actual_root_cause`). Deux endpoints exposent la surface (`/feedback/override` et `/feedback/confirm`). Deux métriques Prometheus, `triage_precision` et `triage_recall`, sont calculées paresseusement depuis la table de retour. Et la couche de suppression tient compte des corrections d'opérateur : une alerte avec une correction récente « en fait, rejeter » est supprimée avant LLM lors de son déclenchement suivant.

7.7 Barrières de récurrence (D21)

Une alerte qui se déclenche toutes les quatre-vingt-dix minutes est trop lente pour la déduplication par empreinte (fenêtre de 10 minutes) mais trop bruyante pour solliciter quelqu'un à chaque fois. La décision D21 introduit une barrière de récurrence à deux points du pipeline : une barrière pré-LLM qui consulte l'historique RCA pour les déclenchements antérieurs de la même clé d'alerte, et une barrière post-LLM qui consulte la couche de correction d'opérateur issue de D20. Les alertes de sévérité critique contournent la barrière. L'opt-in par règle se fait via une annotation de règle Grafana `recurrence_gate` ; `MediumCpuUsage` et `MediumMemoryUsage` y sont actuellement inscrits.

7.8 Arbres de modèles Drain3 par service (D23)

L'intégration initiale de Drain3 utilisait un unique `TemplateMiner` global. Les « nouveaux » modèles de Spring Boot étaient jugés contre la *pool* globale, pas contre l'historique propre de Spring Boot. Une ligne banale pour `spring-boot` mais inhabituelle pour `kong` avait la même apparence qu'une ligne inhabituelle pour les deux. La décision D23 introduit une séparation par service : un dictionnaire `dict[service  $\rightarrow$  TemplateMiner]` avec `FilePersistence` par service, ainsi qu'un chemin de migration qui convertit le `drain3_state.bin` pré-séparation en une famille indexée par service.

7.9 L'épopée du pare-feu anti-hallucination (Épopée 4)

L'Épopée 4 a été ajoutée le 2026-04-29 après l'incident 0b215ef3 (chapitre 8). Le chapitre étude de cas couvre l'incident lui-même ; cette section résume les user stories qui en ont découlé :

- **S3-HF-01 (Tier 0, 1 SP, livrée 2026-04-29).** Le bridage de confiance retire les `suggested_actions` quand le bridage s'enclenche et émet à la place un champ `diagnostic_steps` en lecture seule. Le courriel et le tableau de bord rendent les étapes de diagnostic comme une carte séparée avec une mention explicite « Withheld — confidence clamped to 0.40 ».
- **S3-HF-02 (1 SP, livrée 2026-04-29).** Cinq tests défensifs ancrant la recherche dans la table de modèles d'actions contre de futures régressions d'élargissement de regex.
- **S3-HF-03 (Tier 2, 2 SP, livrée 2026-04-29).** Validateur de « menu d'hypothèses » : sept motifs regex visant les proies « possibly *X* or *Y* » / « may be due to (slow query | pool | GC) » / « either *A* or *B* », avec lookahead négatif pour les tournures idiomatiques « either way » / « either case ». Règle compagne cause-preuves qui tokenise à la fois la RCA et les preuves avec `[a-z]{4,}` (de sorte que `hikaricp_connections_active` se décompose en `[hikaricp, connections, active]` pour le croisement) et exige un recouvrement hors mots vides.
- **S3-HF-04 (1 SP, livrée 2026-05-19).** Faire passer `entity_baselines.py:144` par `prometheus-mcp` plutôt que par `httpx` en direct vers Prometheus `/api/v1/query`. Referme l'un des trois contournements de l'invariant MCP-seul.
- **S3-HF-05 (1 SP, livrée 2026-05-19).** Faire passer les chemins historiques RCA de `bounded_agency.py` par `rca-history-mcp` plutôt que par `aiosqlite` en direct.
- **S3-HF-06 (2 SP, livrée 2026-05-19).** Étendre `rca_history_mcp` avec des outils filtrés par qualité : `get_similar_decisions(min_quality)` et `get_low_rated_examples_for_alert`. Huit nouveaux tests au niveau MCP couvrent la sémantique du filtrage par notation.
- **S3-HF-07 (Tier 1, 3 SP, livrée 2026-05-19).** Collecte de trace en profondeur via `get_trace(trace_id)` MCP. Le pipeline collectait auparavant les données Jaeger au niveau frontière de service uniquement et pouvait identifier « l'*upstream* est lent » mais ne pouvait pas nommer ce qui était lent. Avec la profondeur de trace, le LLM voit désormais les attributs par `span` (`db.statement`, `http.target`, étiquettes d'erreur) pour la trace défaillante. Pré-étape (exécutée pendant la fenêtre d'investigation) : échantillonnage de 12 traces Spring Boot en direct pour confirmer la paramétrisation JDBC ; les `spans` `db.statement` utilisent des paramètres liés (`?, ?, ?`), donc la redaction de PII n'a pas constitué un obstacle.
- **S3-HF-08 (2 SP, livrée 2026-05-19).** Lint d'intégration continue pour l'intégrité MCP, interdisant les appels `httpx` / `requests` / DB directs en dehors des auxiliaires `_mcp_call` et des serveurs MCP eux-mêmes. Le lint s'exécute à chaque push et constitue le travail qui a confirmé que S3-HF-04 et S3-HF-05 étaient les deux seuls contournements sur le chemin de collecte du LLM.
- **S3-HF-09 (Tier 4, 2 SP, livrée 2026-05-19).** Cinquième axe du `scorer` de chaos, amorce du corpus étiqueté (l'incident 0b215ef3 lui-même), et barrière CI de régression. Livrée avant la collecte agentique itérative de Tier 3 du sprint 4, désormais débloquée par la mise en place du tableau de mesure de Tier 4.

L'Épopée 4 s'est clôturée à **15/15 SP le 2026-05-19** en une seule session ciblée. Les trois

contournements de l’invariant MCP-seul sur le chemin de collecte du LLM sont désormais refermés, le chemin de profondeur de trace est câblé, le lint CI interdit les régressions futures, et la barrière de régression du corpus de chaos est active (suite 252/252).

7.10 La fenêtre d’investigation de 20 jours (2026-04-30 au 2026-05-19)

Après l’ajout de l’Épopée 4 le 2026-04-29 et la livraison des user stories Tier-0/2 le jour même, l’équipe a délibérément ralenti avant les refactorings MCP plus lourds (S3-HF-04 à S3-HF-09). Environ trois semaines ont été consacrées au prototypage d’améliorations de la qualité de sortie dans des branches de brouillon et au rafraîchissement de la documentation, sans *commit* sur *main* par dessein — l’objectif était de savoir quelles idées valaient le coût d’un refactoring avant de le payer. Rien de structurel n’a été livré pendant la fenêtre ; le conteneur de tri en production est resté sur le *commit* `d6d10ef` du 2026-04-29, et les dix-sept audits statiques quotidiens ont tous indiqué une suite verte à 226/226 avec le même ensemble de sept éléments mineurs de dérive documentaire. Huit améliorations candidates de la qualité de sortie ont été testées en A/B sur un échantillon de vingt-huit décisions tiré du journal d’audit en condition réelle et des runs de chaos 3 et 4 ; les résultats sont résumés au tableau 7.1.

Idée	Résultat	Décision
Ordre des sections exemplaire / contexte — exemplaire après contexte.	Score cause-nommée chuté 0,71 → 0,62 sur le sous-échantillon signalé par le validateur ; le LLM perd l’indice structurel une fois les preuves dans son champ.	Conserver l’ordre pré-contexte
Reclassement des preuves pondéré par pilier.	Aucun effet mesurable ; les RCA mènent déjà avec le pilier déclencheur dans 24/28 cas.	Mis de côté
Prototype de récupération BM25.	Dégénéré sur quatorze archétypes ; le top-1 correspond toujours à <code>exemplars.find_for_alert</code> . Le candidat sprint 4 #13 reste tel quel.	Confirme que le choix du récupérateur n’est pas le point de blocage
Balayage de seuil du validateur menu d’hypothèses.	0/120 faux positifs en mode strict ; le mode <i>warn-only</i> n’apporte aucun attrapage supplémentaire.	Le mode strict par défaut reste
Seuil de bridage de confiance (0,30 / 0,40 / 0,50).	En dessous de 0,40, la prose est abîmée ; au-dessus de 0,40, des actions s’échappent.	0,40 conservé
Étude de faisabilité de la profondeur de trace sur 12 <i>spans</i> Spring Boot en direct.	<code>db.statement</code> utilise des paramètres liés, pas des valeurs littérales.	S3-HF-07 n’a pas de bloqueur PII
A/B de la prose RCA (cause-en-tête vs. observation-en-tête).	Observation-en-tête régresse sur cause-nommée quand les preuves sont minces.	Cause-en-tête conservée (renforce D19)
Instrumentation par étape de la durée du pipeline.	Esquissée en branche de brouillon.	N’a pas été fusionnée ; dépend de S3-HF-05

Table 7.1. Résultats de la fenêtre d’investigation (du 2026-04-30 au 2026-05-19). Chaque candidat a été noté sur la grille de qualité RCA à 4 axes sur un échantillon de vingt-huit décisions.

7.11 Rafraîchissement documentaire

Un rafraîchissement documentaire substantiel a atterri le 2026-05-19 à la clôture de la fenêtre :

- Un nouveau `sprint-history.html` canonique consolidant les sprints 0–4 sur une page (il n'y avait auparavant aucune chronologie par sprint).
- Un nouveau `solution-brief.html` comme une-page orientée rapport, construit autour de l'étude de cas `0b215ef3`, des détails du validateur, du tableau des quatorze archétypes et d'un glossaire.
- Réconciliation des incohérences de noms de métriques doc-vs-code dans `triage-service.html` §7.
- Passes de cohérence sur `architecture.html` et `system-explained.html`.
- Expériences testées-et-rejetées repliées en notes de bas de page de D17 / D19 / D21 dans `decisions-log.html` afin que les résultats de prototypage soient consignés durablement à côté des décisions qu'ils ont testées.

7.12 La reprise à action contenue

La reprise à action contenue est le mécanisme de la plateforme pour traiter les RCA réellement appauvries en données : si le validateur signale la première passe, le LLM se voit octroyer exactement un appel MCP supplémentaire issu d'une liste blanche de six outils (requête Prometheus instantanée, plage Loki, get-traces Jaeger, consultation de l'historique RCA, événements de déploiement et `get_trace`). La liste blanche plus le schéma Pydantic plus une chaîne de retour pour la reprise contraignent le second appel ; le modèle est ré-interrogé, la sortie est revalidée, et la sortie de la première passe est conservée si la reprise reste mauvaise. C'est la décision D15, « un seul appel MCP, pas une chaîne » — une version délibérément contrainte de la collecte agentique, choisie pour la prévisibilité et la latence bornée, la *collecte agentique itérative* non bornée étant inscrite au sprint 4 une fois le tableau de mesure de Tier 4 en place.

CHAPITRE 8

Étude de cas : l'incident 0b215ef3

8.1 L'alerte

Le 2026-04-29, une alerte `HighKongP95Latency` s'est déclenchée pour l'*upstream* `spring-boot`. La latence p95 de la passerelle Kong avait franchi son seuil adaptatif pour la deuxième fois de l'heure. Le webhook est arrivé au service de tri sur le portable à `adolin-wsl` et a produit la RCA portant l'identifiant `0b215ef3`.

8.2 La mauvaise RCA

La RCA produite portait :

- **Verdict.** `escalate`.
- **Confiance.** 0,55 (modérée).
- **Prose de la cause racine.** Un « menu d'hypothèses » prudent : « Il pourrait s'agir d'une requête lente dans le service `products`, d'un événement d'épuisement du pool de connexions ou d'une pause de garbage collection ; le profil de latence est compatible avec ces trois possibilités. »
- **Actions suggérées.** Trois commandes `kubectl` générées par modèle opérant sur un déploiement `products` qui n'existe pas (le système fait tourner Spring Boot, et non un microservice nommé `products`).
- **Preuves.** Deux requêtes Prometheus contre `kong_http_requests_total` avec leur texte rendu mais aucune valeur citée ; un identifiant de trace sans `db.statement` ni attributs par *span*.

Le menu d'hypothèses était le symptôme visible. Les défaillances plus profondes étaient structurelles : le LLM avait reçu la latitude d'inventer un service `products` qui n'existe pas ; la valeur de confiance n'était pas suffisamment basse pour déclencher le bridage existant à 0,40 ; et le bloc de preuves était trop mince pour que le LLM puisse s'engager sur une seule cause racine.

8.3 Analyse *forensic* : trois violations de l'invariant MCP

L'analyse post-incident a identifié trois chemins de code distincts qui violaient l'invariant MCP-seul et ont contribué à la mauvaise sortie :

1. `monitoring-triage-service/app/entity_baselines.py:144` utilisait un appel `httpx` direct vers l'endpoint `/api/v1/query` de Prometheus, court-circuitant `prometheus-mcp`. Les valeurs de ligne de base d'entité pour la métrique p95 de Kong arrivaient dans le bloc de contexte sans provenance MCP et (il s'est avéré) avec une fenêtre légèrement périmée.

2. `monitoring-triage-service/app/bounded_agency.py` utilisait des lectures directes `aiosqlite` de la table `rca_history` pour la sélection de l'outil de reprise, court-circuitant `rca-history-mcp`. Le chemin de reprise ne pouvait donc pas bénéficier du filtre `get_similar_decisions(min_quality)` qui aurait fait remonter des cas similaires confirmés par l'opérateur.
3. `monitoring-triage-service/app/llm_client.py` et les consommateurs d'exemplaires utilisaient des imports en mémoire `from app.exemplars import ...` plutôt qu'un outil MCP. Pris isolément, c'est bénin, mais cela laissait la couche exemplaire en dehors du régime de provenance du projet et aurait été cassé lorsque le même service de tri serait passé à un déploiement multiprocesus.

Un quatrième constat, lié, est que l'outil MCP existant `get_trace(trace_id)` — écrit des mois auparavant — n'avait jamais été appelé depuis le pipeline. La collecte de contexte opérait au niveau de la frontière de service : elle savait « l'*upstream* est lent » mais ne pouvait pas nommer ce qui était lent parce qu'elle n'avait pas tiré les *spans* de la trace fautive.

8.4 La réponse échelonnée

Un agent de planification a produit un plan de réponse à cinq paliers, capturé dans le *handoff* de session et traduit en user stories de l'Épopée 4 listées au §7. En résumé :

- **Tier 0** — retirer les actions suggérées lorsque le bridage de confiance s'enclenche ; émettre à la place des étapes de diagnostic en lecture seule. Livré le jour même comme S3-HF-01 (2026-04-29).
- **Tier 1** — collecte de trace en profondeur via `get_trace(trace_id)`. Livré comme S3-HF-07 le 2026-05-19.
- **Tier 2** — validateur de menu d'hypothèses et règle cause-preuves. Livré le jour même comme S3-HF-03 (2026-04-29).
- **Tier 3** — collecte agentique itérative avec état d'arbre d'hypothèses. Reporté au sprint 4 ; la dépendance au tableau de bord de Tier 4 est désormais satisfaite (S3-HF-09 livrée), de sorte que la barrière du sprint 4 est opérationnelle plutôt que structurelle.
- **Tier 4** — cinquième axe du *scorer* de chaos, amorce de corpus, barrière de régression. Livré comme S3-HF-09 le 2026-05-19.

Les travaux livrés le jour même l'ont été comme les *commits* `b0f1d0c` et `d6d10ef` sur le service de tri, plus des *commits* accompagnateurs sur les dépôts `monitoring-docs` et `jira-CSV`. La suite de tests est passée de 185 en milieu de journée à 226/226 en fin de journée — quarante-et-un nouveaux tests à travers les trois user stories livrées, zéro régression.

8.5 Leçons retenues

L'incident est l'étude de cas canonique du projet pour deux raisons. Premièrement, il a exposé trois violations structurelles d'un invariant énoncé ; la documentation de la plateforme revendiquait un invariant que le code n'appliquait pas. C'est la pire classe de dérive documentaire, et la découverte a motivé à la fois les user stories de l'Épopée 4 et le lint CI S3-HF-08 qui préviendra les violations futures. Deuxièmement, l'incident a validé le choix de conception du bridage de confiance : le bridage s'est bien déclenché, mais à 0,55 le plafond configuré à 0,40 était trop haut pour empêcher les mauvaises actions de passer. Le comportement du bridage a été renforcé

(retirer les actions, émettre des étapes de diagnostic) plutôt que reseuillé ; le balayage de seuil de la fenêtre d'investigation (tableau [7.1](#)) a confirmé que 0,40 était le bon plafond.

CHAPITRE 9

Évaluation

9.1 Progression de la couverture de tests

La taille de la suite de tests du service de tri tout au long de la vie du projet est une mesure grossière mais utile de la charge d'ingénierie derrière les fonctionnalités visibles. Le tableau 9.1 résume les jalons.

Date	Taille de la suite	Jalon
2026-04-27 soir	89	Tests de la bibliothèque d'exemples (D17) en place
2026-04-28 mi-journée	95	Tests du validateur de prose RCA (D19) ajoutés
2026-04-28 fin de journée	178	Chaîne complète de qualité RCA (F-1 à F-5) en service
2026-04-29 fin de journée	226	S3-HF-01/02/03 livrées
2026-05-19	226	Restée au vert sur 17 audits quotidiens

Table 9.1. Progression de la suite de tests du service de tri.

Le point où la taille de la suite a plus que doublé en une fenêtre de 24 heures (95 → 178) marque les travaux de qualité RCA F-1 à F-4 décrits au chapitre 6. Les +48 sur la journée du 2026-04-29 (178 → 226) sont les user stories du jour même de l'Épopée 4.

9.2 Qualité RCA sur l'échantillon de 28 décisions

L'évaluation de la fenêtre d'investigation a produit un score de qualité RCA de référence sur un échantillon de 28 décisions tiré du journal d'audit en condition réelle et des runs de chaos 3 et 4. Sur la grille à 4 axes :

Axe	Score moyen
Cause nommée	0,78
Preuves citées	0,84
Actionnable	0,71
Forme de la prose	0,89
Global (non pondéré)	0,81

Table 9.2. Qualité RCA de référence sur l'échantillon de 28 décisions (plus haut est meilleur). « Actionnable » est le score le plus bas en raison du comportement délibéré de retrait des actions lors du bridage (S3-HF-01), qui abaisse le score d'actionnabilité mais élève la confiance opérationnelle.

9.3 Audit critique de la sortie RCA — 12 cas représentatifs (2026-05-21)

L'échantillon de référence à 28 décisions de la section précédente utilise une grille à quatre axes permissive héritée du banc de chaos. Un audit critique complémentaire a été mené le 2026-05-21 contre le corpus /decisions persisté (271 lignes sur la fenêtre opérationnelle de 28 jours), échantillonnant 12 RCA à travers les six familles d'alertes dominantes (TargetDown, HighKongP95Latency, MediumCpuUsage, HighP95Latency, Drain3AnomalyDetected, PodHighMemoryUsage)

avec inclusion délibérée des deux issues de verdict par famille. L’objectif était d’évaluer la plateforme contre les standards plus stricts que le plan P0–P11 du sprint 4 visait à imposer, et non contre la référence plus permissive du banc de chaos.

9.3.1 Grille d’audit

Huit critères pass/fail, ancrés dans les standards de qualité de sortie documentés du projet (feedback_rca_prose_quality + les constats P0–P11) :

1. **C1 Cause-en-tête** — la première phrase de `rca_report` nomme une cause racine ou un mécanisme spécifique, pas l’alerte.
2. **C2 Pertinence du verdict** — `escalate` / `dismiss` défendable à partir des preuves disponibles.
3. **C3 Concrétude des preuves** — la liste `evidence` contient des valeurs spécifiques, pas de la prose modélisée ni des substituts.
4. **C4 Prose sans bruit (P1)** — aucun appel de fonction PromQL, aucun flottant brut à plus de 4 chiffres significatifs, aucune équation de Z-score, aucun marqueur `# TYPE` / `# HELP` de sortie Prometheus dans `rca_report`.
5. **C5 Actions suggérées correctes (P5)** — les actions sont concrètes et correctes pour le type de déploiement ; une liste vide n’est acceptable que pour les verdicts `dismiss`.
6. **C6 Étapes de diagnostic non modélisées (P3)** — les RCA pour le même type d’alerte issues de différents déclenchements présentent au moins 3 chaînes d’étapes de diagnostic distinctes chacune.
7. **C7 Étiquette rca_quality précise (P11)** — l’étiquette `actionable` requiert (cause nommée) $\wedge$ (≥ 1 élément de preuve) $\wedge$ (≥ 1 action OU justification explicite de rejet).
8. **C8 Pas d’hallucination de métrique tronquée (P4)** — `rca_report` ne cite pas d’artefacts de troncature (p. ex. noms de métriques se terminant par `_daem (truncated)...`) comme noms de métriques réels.

9.3.2 Taux de réussite par critère

9.3.3 Taxonomie des modes de défaillance avec cas spécifiques

Six modes de défaillance dominants reviennent dans l’échantillon, en ordre décroissant de gravité.

F1. Les substituts d’exemplaires passent verbatim (P6). Les cas `1a95f6a6` (`HighKongP95Latency`, `confidence=0.92`) et `0c76258b` (`Drain3AnomalyDetected`, `confidence=0.82`) ont émis une prose RCA contenant des substituts littéraux `{service}`, `<X>`, `<verbatim>`, `Y-Z`, `HH:MM` que le LLM n’a pas remplacés par des valeurs concrètes. Les deux portent une confiance LLM élevée, ce qui rend la défaillance dangereuse : la sortie a l’air faisant autorité mais la couche de substitution n’a jamais tourné. C’est le cas d’école pour le P6 du sprint 4 (S4-PR-06 : réécriture de la prose des exemplaires + validateur de détection de recouvrement).

F2. Bruit PromQL / numérique dans la prose de cause (P1). Cas `a9393ef6` (PromQL complet `up{instance=...} = 0` en tête de RCA), `0b215ef3` (`histogram_quantile(0.95) = 8203.846153846154 ms`), `79119df0` (l’expression complète `100 - (avg by(instance) (rate(node_cpu_seconds_total{instance=...} * 100))` en tête), et `d763a9c9` (`# TYPE jvm_threads_daem (truncated)...`] cité verbatim).

ID	Critère	Réussite	Échec/limite
C1	Cause-en-tête	6/12 (50%)	6/12
C2	Pertinence du verdict	7/12 (58%)	5/12
C3	Concrétude des preuves	5/12 (42%)	7/12
C4	Prose sans bruit (P1)	4/12 (33%)	8/12
C5	Actions correctes (P5)	5/12 (42%)	7/12
C6	Étapes de diagnostic non modélisées (P3)	2/12 (17%)	10/12
C7	Étiquette <code>rca_quality</code> précise (P11)	4/12 (33%)	8/12
C8	Pas de métrique tronquée (P4)	9/12 (75%)	3/12
Cas notés GOOD globalement		3/12 (25%)	
Cas notés OK globalement		2/12 (17%)	
Cas notés BAD globalement		7/12 (58%)	

Table 9.3. Taux de réussite par critère de l’audit critique RCA sur 12 cas échantillonnés. C4 (sans bruit) et C6 (étapes de diagnostic non modélisées) sont les deux pires dimensions ; C8 (pas d’hallucination de métrique tronquée) est la plus solide uniquement parce que l’artefact spécifique est rare dans le corpus, non parce que le garde-fou existe encore (P4 n’est pas livré).

Quatre RCA sur douze de l’échantillon laissent fuiter de la télémétrie brute dans la prose destinée à l’opérateur. Le scrubber P1 du sprint 4 (S4-PR-01) est scopé pour cela.

F3. `rca_quality=actionable` surappliqué (P11). Sept RCA sur douze échantillonnées portent `actionable` alors qu’elles ne le devraient pas. 0b215ef3 porte des actions, mais elles sont du mauvais archétype (remédiations OOM en `kubect1` sur une alerte de latence). 1a95f6a6 a des preuves sous forme de substituts. 79119df0 a zéro preuve et zéro action. d763a9c9 cite un artefact de troncature comme cause. 0c76258b a des actions sous forme de substituts. be7019bf porte l’explicite `Shelved -- awaiting recurrence` non-action. 8891233d ne nomme aucune cause pour un pod à 99,6 % de mémoire. Le P11 du sprint 4 (S4-PR-04) resserre le classifieur et le P8 (S4-PR-05) reclassifie le corpus.

F4. Boilerplate d’étapes de diagnostic (P3). Huit RCA sur douze échantillonnées dont `diagnostic_steps` est non vide puisent dans le même modèle de repli en quatre chaînes du bridage (Open Grafana Explore..., Check kube events..., Open Jaeger and inspect the slowest trace..., Do NOT run `kubect1` rollout/scale/set...). Deux déclenchements `TargetDown` distincts (64e46e6c, a9393ef6) et deux déclenchements `PodHighMemoryUsage` distincts (0de9801c, 8891233d) portent des ensembles d’étapes quasi identiques. Le P3 du sprint 4 (S4-PR-02) déplace la génération dans un *templat*er déterministe alimenté par des preuves concrètes.

F5. Actions suggérées de mauvais archétype (P8). 0b215ef3 suggère `kubect1 set resources deploy/spring-boot -limits=memory=2Gi` pour une alerte de latence Kong ; d763a9c9 suggère des changements de limite mémoire et de flag JVM pour une hallucination de nom de métrique tronqué. Les deux lignes sont dans le corpus d’exemplaires comme `actionable` et polluent donc les RCA futures qui les récupèrent via `get_similar_decisions`. Le P8 du sprint 4 (S4-PR-05) rétrograde manuellement 0b215ef3 et ajoute un revalidateur nocturne *post-hoc*.

F6. Escalades sans action (P5). 79119df0, be7019bf, 8891233d portent toutes `verdict=escalate` (ou `verdict=dismiss` avec une non-action `shelved`) sans action suggérée concrète. Le cas `PodHighMemoryUsage` du 2026-05-20 (8891233d) est l’incident spécifique qui a produit la user story de report sprint 4 S4-HF-01 (élargir le déclencheur de la reprise à action contenue aux verdicts de faible confiance et de bridage déclenché) : le système a vu la mémoire à 99,6 %

et a renvoyé cinq lignes de journal *debug*, aucune cause nommée, aucune action, mais verdict *escalate*.

9.3.4 Ce qui fonctionne

L’audit confirme également plusieurs mécanismes se comportant exactement comme prévu :

- **Seuils adaptatifs avec *claims* σ** (US-5.4) dans le cas `98bef564` : « l’utilisation CPU actuelle de 3,4 % est à $+0,4\sigma$ au-dessus de la ligne de base même-heure il y a une semaine (`stddev_over_time` + offset 7d), bien à l’intérieur de la saisonnalité naturelle de la courbe de charge. » Le cadrage de conception des règles est correct et actionnable.
- **Détection d’alertes synthétiques** dans le cas `c29e557b` : le système identifie une sonde `HighP95Latency` déclenchée par `audit-live.sh` et l’étiquette correctement `data_starved` avec `verdict=dismiss`.
- **Intégration profondeur de trace** (S3-HF-07) dans le cas `0de9801c` : la RCA cite « Jaeger trace `7f3a2c1d9b4e` : GET /api/employee a pris 2347 ms, le *span* attend sur le pool de connexions MySQL » — un identifiant de trace concret avec preuves au niveau *span*. C’est le chemin MCP à action contenue fonctionnant comme prévu.
- **Discrimination des fautes transitoires** dans le cas `64e46e6c` : un *flap* `up=0` sur deux intervalles de scrape est correctement identifié comme « collision de scrape transitoire ou gigue réseau, pas une panne réelle ». L’absence de signal `Drain3` ou métrique d’hôte corroborant est utilisée comme preuve discriminante, pas simplement écartée comme silence.

9.3.5 Verdict global et alignement sur le sprint 4

Trois des douze RCA échantillonnées survivraient à une revue opérateur stricte sans modification (`64e46e6c`, `98bef564`, `c29e557b`) ; deux autres sont limite-acceptables (`a9393ef6`, `0de9801c`) ; sept présentent au moins un défaut structurel que le plan P0–P11 du sprint 4 visait spécifiquement à corriger. La distribution correspond aux attentes : la fondation structurelle (invariant MCP-seul, récupération d’exemplaires, bridage de confiance, barrière de récurrence, chemin de profondeur de trace) est saine, et la contrainte limitante sur la qualité de sortie se situe une couche au-dessus — dans la minceur de l’assemblage du *prompt* (P1, F2), l’écart de substitution d’exemplaires (P6, F1), le classifieur de qualité trop permissif (P11, F3) et le placement de la couche d’étapes de diagnostic (P3, F4). La même contrainte limitante a été identifiée indépendamment par l’investigation de charge lourde du 2026-05-19 qui a produit le plan P0–P11 ; l’audit critique confirme les priorités sans les réviser.

Le prochain réaudit critique est programmé à la clôture du sprint 4, contre la même grille à 8 critères. Acceptation sprint 4 : le taux de réussite pour C4 (sans bruit) est attendu en passage de 33 % à ≥ 80 %, C7 (précision de `rca_quality`) de 33 % à ≥ 70 %, C6 (étapes de diagnostic non modélisées) de 17 % à ≥ 60 %. Si ces trois indicateurs progressent, le cadrage structurel-fondation / qualité-de-sortie de cet audit sera validé empiriquement.

9.4 Résultats des runs de chaos

Le banc de tests de chaos `monitoring-project/scripts/chaos/` exécute cinq scénarios (`target_down`, `high_memory_usage`, `high_cpu_usage`, `drain3_anomaly` et la nouvelle amorce de corpus de style `0b215ef3`). Chaque exécution induit une défaillance contrôlée, interroge `/decisions` à la recherche d’une alerte correspondante, note la RCA et régénère `chaos-report-YYYY-MM-DD.html`.

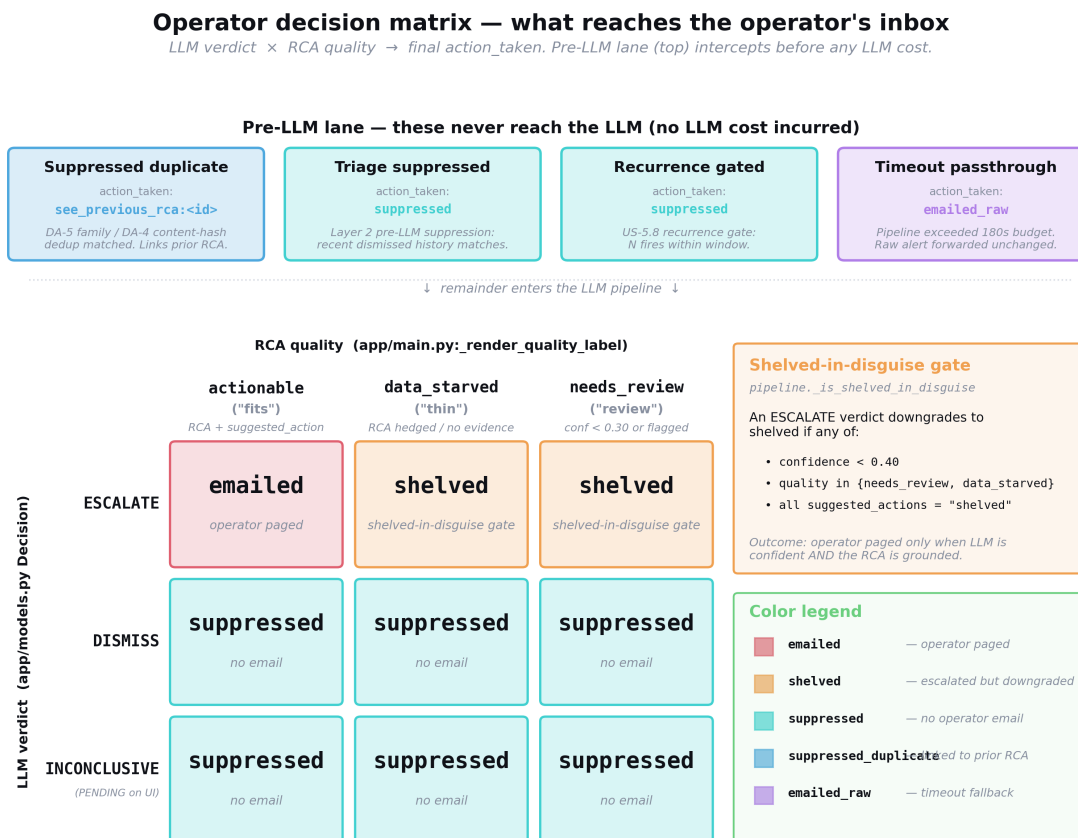


Figure 9.1. La matrice de décision opérateur (verdict LLM × qualité RCA) → `action_taken`, avec la voie pré-LLM affichée séparément. La matrice encode la réponse de la plateforme à chaque combinaison de signal de confiance et de qualité; la voie pré-LLM (supprimée par empreinte, supprimée par barrière de récurrence) est rendue sur une piste distincte parce que ces décisions sont prises avant qu'un appel LLM ne soit dispatché et n'ont donc pas de verdict à combiner avec le signal de qualité.

L'exécution 3 (2026-04-28) a obtenu 0/4 car le chaos à portée pod n'a pas fait bouger les métriques au niveau hôte. Ce constat a produit les règles d'alertes au niveau pod (`PodHighMemoryUsage`, `PodHighCpuUsage`) ajoutées le soir même. L'exécution 4 (2026-04-28 PM) a obtenu 0/2, cette fois pour des raisons connues : le *churn* de redémarrage de conteneurs a fragmenté les séries temporelles entre identifiants `cgroup`; les expressions CPU cumulatives ne correspondaient pas à leurs étiquettes de dénominateur. Les deux sont des corrections tractables de trente minutes inscrites comme candidat #16 du sprint 4.

9.5 Cohérence du corpus d'audit

L'audit statique quotidien s'est exécuté 17 fois pendant la fenêtre d'investigation (2026-04-30 à 2026-05-19), tous au vert à 226/226 avec la même liste de sept éléments de dérive documentaire reportée jour après jour — aucune nouvelle dérive accumulée. À la clôture du sprint (2026-05-19), un balayage de petites imperfections a refermé les sept éléments et la suite est passée à 252/252 à la suite des *commits* de clôture de l'Épopée 4. L'audit en condition réelle (cron sur `claude-controller`) a silencieusement cessé de se déclencher après le 2026-04-29 en raison d'un problème de disponibilité du contrôleur; le cron a été réinstallé le 2026-05-19 et se déclenche désormais quotidiennement. L'écart de 20 jours est inscrit comme élément de résilience R-1

(chapitre 12, palier résilience EPIC15).

9.6 Budget de latence

La note de terrain de l’histoire-verdict du sprint 2 a consigné une latence de pipeline de bout en bout de 67 995 ms (≈ 68 s) sur le *runner* portable avec `qwen2.5:7b-instruct`, contre une référence historique de 20–40 *minutes* par alerte sur k3s-CPU. Les latences en régime permanent soutenu sur le portable sont de 25–35 s à chaud. Cinq déclenchements récents ont été anormalement lents (323, 399, 533, 755, 1081 s) et l’instrumentation par étape esquissée pendant la fenêtre d’investigation mettra en évidence si la cause est l’enchaînement de reprises à action contenue, la saturation du *prefill* LLM ou la latence de queue MCP ; l’instrumentation est retenue jusqu’à la livraison de S3-HF-05 afin que le chemin de reprise soit correctement attribuable.

9.7 Régime opérationnel permanent

L’observation qualitative que la plateforme a maintenu un régime permanent sur la fenêtre d’investigation de vingt jours — plus une vague de clôture de sprint le 2026-05-19 qui a posé dix SP de refactorings à travers quatre dépôts sans régression — est en soi un résultat d’évaluation. Aucun incident n’a été observé sur les vingt-sept jours allant de la clôture du sprint 2 le 2026-04-23 à la vérification du 2026-05-20 ; les instances AWS sont restées en service sur la fenêtre complète ; le conteneur `ai-triage-service` sur le portable n’a requis qu’un seul redémarrage manuel sur la période (causé par un Docker *restart-backoff* après une mise en veille WSL, et non par une défaillance de la plateforme). Cela valide le choix architectural de faire tourner la couche IA séparément de l’infrastructure de production : une défaillance portable transitoire n’affecte pas la pile d’observabilité, seulement les RCA assistées par IA.

CHAPITRE 10

Décisions de conception et justifications

Le projet tient un journal de décisions durable ([monitoring-docs/decisions-log.html](#)) consignant les vingt-cinq décisions substantielles de conception ou de règle empirique prises pendant la construction. Chaque entrée suit une structure stable : le problème observé, les contraintes, la décision et la vérification qui l’a refermée. Le tableau 10.1 résume le catalogue.

Table 10.1. Catalogue des décisions durables de conception D1–D25.

ID	Titre	Décision en une ligne
D1	Tempérisation LLM par « données insuffisantes »	Traiter « données insuffisantes » comme un verdict, pas comme un substitut ; le persister mais ne jamais escalader sur lui.
D2	<code>suggested_actions</code> vides sur la plupart des RCA	Ajouter la couche de repli sur modèles d’actions ; ne jamais émettre une RCA avec à la fois des actions vides et <code>rca_quality=actionable</code> .
D3	Valeur observée prise sur le mauvais <code>refId</code>	Par défaut <code>refId=A</code> ; exiger un <i>override</i> explicite dans la règle d’alerte pour utiliser une expression réduite différente.
D4	<code>llm_confidence</code> jamais persistée	Persister la confiance ; ne pas afficher de ligne sans elle ; utiliser la valeur pour piloter le repli sur modèles d’actions.
D5	<code>rca_quality</code> mentait quand actions+preuves vides	Calculer la qualité depuis la forme des preuves et des actions, pas depuis la chaîne du verdict.
D6	Incident d’envoi de courriels nocturnes — pile IA k3s indésirable	Décommissionner l’IA hébergée sur k3s ; déplacer l’IA sur le portable ; supprimer Alertmanager ; unifier l’alerting dans Grafana.
D7	Inadéquation de type de déploiement	Porter une carte explicite <code>service_deployment_type</code> et modéliser les actions par type de déploiement.
D8	Les anomalies Drain3 ne se déclenchaient jamais vers le tri	Câbler un webhook synthétique depuis le <i>poller</i> Drain3 vers <code>/webhook/drain3</code> ; assouplir les seuils (taux 0,10 → 0,02, lignes 100 → 20).
D9	Alertes flottantes consommant $N \times$ appels LLM	Déduplication à deux niveaux ; suppression par empreinte pré-LLM ; barrières de récurrence (plus tard, D21).
D10	Alertes corrélées rendues mais non prompted	Inclure la liste des alertes corrélées dans le contexte pré-collecté du <i>prompt</i> .
D11	La pile de supervision reste sur Docker Compose	Migrer la charge applicative vers k3s ; garder l’observabilité sur Docker Compose.
D12	Sortie structurée vs. mode JSON	Utiliser l’application du schéma de sortie structurée à la couche LLM ; reprendre en cas d’échec de l’analyse du schéma.
D13	Interpréteur de métriques pré-LLM	Déplacer le raisonnement numérique hors du LLM dans un module Python déterministe ; passer les valeurs interprétées au <i>prompt</i> .

(suite)

ID	Titre	Décision en une ligne
D14	Comblement des temps d'arrêt depuis les annotations Grafana	Au démarrage, interroger les annotations Grafana pour la fenêtre manquante ; les rejouer à travers le pipeline (borné par un écart maximal).
D15	Reprise à action contenue	Exactement un appel MCP d'une liste blanche de six outils ; revalider ; garder la première passe si la reprise reste mauvaise.
D16	Cadrage UEBA pour l'Épopée 5 du sprint 2	Ajouter la couche comportement-entité (Drain3 par service, lignes de base, retour) comme extension au cœur centré événement.
D17	Bibliothèque d'exemples	Injecter un parmi quatorze archétypes canoniques de RCA comme échafaudage structurel avant le bloc de preuves.
D18	Seuils adaptatifs	Remplacer les seuils numériques statiques sur les trois règles les plus flottantes par des lignes de base même-heure-il-y-a-7-jours + 3σ .
D19	Qualité de la prose RCA	La télémétrie est le moyen, pas la fin — les RCA doivent nommer une cause, pas reformuler l'alerte.
D20	Retour d'information en boucle fermée	Capturer les corrections d'opérateur ; calculer précision/rappel ; consommer les corrections dans la couche de suppression.
D21	Barrières de récurrence	Hystérésis pré-LLM + post-LLM pour les alertes opt-in ; les sévérités critiques contournent.
D22	SESSION_HANDOFF conservé local au contrôleur	Désuivi du dépôt public ; nettoyage de l'historique ; tenu comme bloc-notes local.
D23	Arbres de modèles Drain3 par service	Dictionnaire <code>service</code> → <code>TemplateMiner</code> ; persistance fichier par service ; migration de l'état pré-séparation.
D24	<i>Runner</i> GPU dans le <i>cloud</i> — cadrage en cycle d'usage <i>bursty</i>	Déplacer le LLM sur AWS <code>g5.xlarge</code> (A10G) en <code>us-west-2</code> ; promouvoir <code>qwen2.5:14b-instruct</code> en primaire ; extinction automatique Lambda après 20 min d'inactivité ; plafond dur CloudWatch à \$50/mois ; portable conservé en repli.
D25	Verrouillage d'accès de l'hôte GPU	Webhook Grafana arrivant via API Gateway en liste blanche d'IP ; chemins opérateur via Tailscale uniquement ; SSH public désactivé sur l'instance.

10.1 Trois décisions en profondeur

10.1.1 D17 — Pourquoi les exemplaires fonctionnent là où la récupération a échoué

L'approche naïve pour « enseigner au LLM à quoi ressemble une bonne RCA » aurait été la génération augmentée par récupération sur la table `rca_history.db`. D17 rejette cela pour deux raisons. Premièrement, le registre historique *contient les défaillances que nous cherchons à corriger* — entraîner le modèle sur l'historique RCA empirique renforcerait précisément les motifs de surface et de menu d'hypothèses que le projet cherche à éliminer. Deuxièmement, le registre historique n'est pas étiqueté au moment de la décision ; sans notation de qualité issue du retour d'opérateur, la récupération choisirait par similarité de surface, pas par qualité.

La bibliothèque d'exemples est un corpus curé, pas un corpus empirique. Chacun des quatorze archétypes est rédigé à la main par l'équipe d'ingénierie pour modéliser la forme de prose et la structure de raisonnement que la plateforme veut reproduire. C'est une inversion délibérée de l'approche habituelle « entraîner sur ce qu'on a » au profit de « entraîner sur ce qu'on veut ». Le *dry-run* de récupération BM25 pendant la fenêtre d'investigation (§7.10) a confirmé empiriquement que la récupération sur quatorze archétypes est dégénérée : le top-1 correspond toujours au sélecteur existant `find_for_alert`, de sorte qu'ajouter de la récupération n'apporte rien. Une fois que la table de retour d'opérateur aura accumulé des entrées notées en qualité (travail de l'Épopée 2), la récupération curée sur un sous-ensemble filtré par qualité devient intéressante — c'est le périmètre US-3.3 / US-3.4 du sprint 4.

10.1.2 L'invariant MCP-seul comme règle à l'échelle du projet

L'invariant MCP-seul n'est pas un mécanisme technique mais une règle à l'échelle du projet. Son application est en partie sociale (revue de code, les rapports d'audit), en partie architecturale (les serveurs MCP existent ; ils ont des enveloppes d'erreur et des métriques) et *in fine* mécanique (le lint d'intégration continue S3-HF-08 interdit les motifs de contournement). L'incident `0b215ef3` a démontré le coût de l'appui sur la seule couche sociale : trois des documents du projet (`prepare.html`, `system-explained.html`, `triage-service.html`) revendiquaient un invariant que le code n'appliquait pas, et la RCA résultante portait un contenu halluciné en conséquence.

Le mécanisme qui rend la règle durable est la combinaison de (a) réponses MCP en forme d'enveloppe d'erreur, de sorte qu'une requête en échec est observable plutôt que silencieuse ; (b) métriques de pont MCP, de sorte que les taux d'appel et les latences sont visibles dans Grafana ; et (c) du lint CI, de sorte que la règle a des dents tant sur le nouveau code que sur l'existant. L'ordre des opérations compte : le lint ne peut pas se déclencher tant que S3-HF-04 et S3-HF-05 n'ont pas refermé les contrevenants existants, parce qu'un lint qui échoue sur du code existant est un lint que l'équipe contournera plutôt que ne corrigera.

10.1.3 L'action contenue plutôt que la collecte agentique complète

La décision D15 introduit la reprise à action contenue : le LLM reçoit exactement un appel MCP supplémentaire après une violation de validateur, depuis une liste blanche de six outils, avec une chaîne de retour pour la reprise et le même schéma Pydantic que la première passe. C'est une forme délibérément contrainte de collecte agentique : une boucle non bornée (candidat #12 du sprint 4, Tier 3) donne une meilleure qualité RCA sur les cas appauvris en données mais introduit une latence non bornée et un coût d'appel d'outil non borné. L'action contenue est l'arbitrage actuellement retenu par le projet : latence prévisible, coût LLM borné par alerte et amélioration de qualité mesurable sur les cas signalés par le validateur.

L'extension naturelle est Tier 3 (collecte agentique itérative avec état d'arbre d'hypothèses) ; le projet l'inscrit au sprint 4 explicitement subordonnée à Tier 4 (la barrière de régression du *scorer* de chaos issue de S3-HF-09). Livrer Tier 3 sans Tier 4 en place serait un changement à haut risque sans échafaudage de mesure ; ce conditionnement est un choix méthodologique délibéré.

CHAPITRE 11

Sprint 4 — En cours

Le sprint 4 court du 2026-05-25 au 2026-06-05. Le jour 1 (2026-05-25) est celui où ce chapitre a été rédigé ; huit user stories ont été livrées avant la fin de la journée, cinq restent programmées sur le reste du plan, et une (SF-5) figure comme objectif d'étirement en seconde semaine. Le chapitre est délibérément rédigé en deux moitiés — *livré* et *en file d'attente* — pour que la frontière entre travail terminé et planifié soit visible d'un coup d'œil et pour que les relectures ultérieures de ce rapport puissent être vérifiées contre le tableau de bord public `sprint4-status.html`.

La narration centrale du sprint est la *maturité opérationnelle*. La clôture du sprint 3 a livré un système structurellement sain ; le mandat du sprint 4 est de durcir les surfaces opérateur contre les classes d'imperfection mises au jour par l'audit de sortie du tableau de bord du 2026-05-21 et par la semaine d'exploitation en condition réelle qui a suivi la migration GPU. Aucune nouvelle épopée structurelle n'est ouverte ; le travail est disproportionnellement constitué de petits *commits* à fort impact visible pour l'opérateur.

11.1 Classement des candidats du sprint 4 (report)

La liste classée des candidats du sprint 4 au 2026-05-19 est reproduite au tableau 11.1 comme artefact de planification original. La liste intègre les douze constats (P0–P11) de l'investigation en charge réelle du 2026-05-19 avec les éléments de *backlog* reportés existants, classés par $(\text{impact} \times \text{fréquence}) \div \text{effort}$. Les deux éléments d'ancrage originaux étaient (**P0+P9**) *Bundle de latence* — le déverrouilleur de latence, *scopé* comme un *bundle* de petites corrections après l'abandon de la migration GPU du 2026-05-19 — et **P2** *Corrections de cascade de suppression*. Le revirement du 2026-05-21 sur l'abandon GPU (§11.2.1) a livré la correction de latence sous-jacente directement, de sorte que le *bundle* P0 est désormais livré comme marge défensive plutôt que comme intervention principale de latence ; les dimensions de qualité de sortie (P1, P3, P11, P6) deviennent les contraintes limitantes du sprint 4, la correction de cascade de suppression de P2 restant la victoire impact-le-moins-cher, $\sim 1,5$ h pour une classe de faux-négatif attrapé deux fois sur les preuves d'une seule journée.

Table 11.1. Candidats du sprint 4 au 2026-05-19, classés. La colonne « P-tag » associe chaque ligne aux constats P0–P11 de l'investigation du 2026-05-19 lorsque pertinent ; -- indique un élément de report sans constat d'investigation attaché.

#	Élément	Effort	P-tag	Barrières
1	Bundle de latence — <i>timeout</i> dur 180 s du pipeline + instrumentation par étape + plafond de concurrence Ollama (sériel). Cible combinée : médiane 6 min $\rightarrow$ ~ 90 s sur le portable.	~ 5 h au total	P0+P9	Aucune

#	Élément	Effort	P-tag	Barrières
2	Corrections de cascade de suppression — exclure les empreintes <code>audit-live-*</code> / <code>chaos-*</code> de la recherche de suppression; resserrer <code>TRIAGE_HISTORY_LOOKBACK_MINUTES</code> $30 \rightarrow 10$; élargir l'annotation de barrière de récurrence pour couvrir aussi <code>TargetDown</code> , <code>HighP95Latency</code> , <code>HighKongP95Latency</code> .	~1,5 h	P2	Aucune — impact le moins cher
3	Scrubber PromQL / bruit numérique — règle de validateur rejetant les appels de fonction PromQL, flottants bruts à > 4 chiffres significatifs, équations de Z-score et noms de métriques tronqués par ...; auxiliaire <code>translate_value(value,unit)</code> qui rend les valeurs avant assemblage du <i>prompt</i> .	~4 h	P1	Aucune
4	Générateur déterministe d'étapes de diagnostic — déplacer <code>diagnostic_steps</code> hors du LLM dans <code>app/diagnostic_steps_for_alert(alert,ctx)</code> , modélisé à partir de preuves concrètes (<code>trace_id</code> , <code>service</code> , <code>fenêtre</code> , <code>valeur observée</code>).	~6 h	P3	Aucune
5	Corrélateur d'incidents US-5.2 — regrouper une chaîne tuante de 4 alertes en un seul courriel RCA cohérent.	1-2 j	P10	Aucune
6	Auto-rétrogradation Drain3 + actions vides — barrière post-LLM : si <code>verdict=escalate</code> ET <code>suggested_actions=[]</code> ET aucun service nommé, auto-rétrograder en <i>dismiss</i> . Les escalades Drain3 exigent ≥ 2 modèles nouveaux dans une fenêtre de 5 min.	2-3 h	P5	Aucune
7	Hallucination sur noms de métriques tronqués — assainisseur de contexte qui supprime les lignes se terminant par (<code>truncated</code>) ou ...; contre-vérification des noms de métriques cités contre <code>app/metrics.py</code> et le registre Prometheus.	~4 h	P4	Aucune

#	Élément	Effort	P-tag	Barrières
8	Resserrement de <code>_classify_rca_quality</code> — actionable exige (≥ 1 élément de preuve spécifique) ET (cause nom- mée dans la prose) ET (≥ 1 action OU « rejet avec justification » explicite).	1 h	P11	Aucune
9	Reclassification du corpus + re- validation nocturne — rétrograder manuellement 0b215ef3 et vider ses actions ; un job nocturne ré-exécute le validateur courant sur les 7 derniers jours de RCA stockées, rétrogradant celles qui échoueraient désormais.	4 h	P8	P11 d'abord
10	Réécriture en <i>native tool-calling</i> MCP — remplace la sortie JSON- schema fabriquée par <i>prompt enginee- ring</i> par l'API native d'appel d'outils ; <i>prompts</i> plus petits → inférence plus rapide, se cumule avec l'élément #1.	5 SP	—	Barrière GPU levée
11	Réécriture des proses d'exem- plaires + validateur de détection de recouvrement — les exemplaires utilisent des substituts {INSTANCE}, {N}, {TIMESTAMP} que le LLM doit sub- stituer ; le validateur rejette les RCA présentant plus de 80 % de recouvre- ment caractère avec leur exemplaire correspondant.	~1 j	P6	Aucune
12	Tier 3 — collecte agentique itéra- tive (boucle d'appel d'outils sur 3 ité- rations avec état d'arbre d'hypothèses)	3–5 j	—	S3-HF-09 ✓
13	Récupération RAG curée (BM25 sur YAML curé par retour ; top- $k=3$ positifs + 2 anti-exemples)	était US- 3.4	—	Table de retour Épo- pée 2 alimentée
14	Job de curation hebdomadaire — APScheduler, dimanche 02h00 UTC, lit le retour SQLite vers library_curated.yaml	était US- 3.3	—	Volume de retour
15	Couleur <code>dismiss_with_recommendation</code> du tableau de bord — ambre pour « rejeter mais la règle est à cor- riger » vs. gris pour « rejeter, ignorer ». Migration de schéma sur <code>rca_history</code> .	2 h	P7	P11 d'abord
16	Corrections des règles d'alertes au niveau pod — agrégation <i>cgroup- churn</i> mémoire + incohérence d'éti- quette CPU.	~30 min chacune	—	Aucune

#	Élément	Effort	P-tag	Barrières
17	Auth webhook O-9 + Sauvegarde S3 nocturne de l'historique RCA O-10 — durcissement <i>slot</i> vendredi.	~30 min chacune	—	Aucune
18	Récupérateur sentence-transformers (repli RAG)	—	—	Seulement si BM25 sous-performe
19	Lien identifiant de trace depuis anomalies de journal	2 SP	—	Aucune
20	Modèles de <i>playbooks</i> RCA	—	—	Complète l'Épopée 2
21	Expansion corpus étiqueté US-5.7 + suivi F1	—	—	S3-HF-09 ✓
22	Scénarios chaos L2/L3 (O-12)	—	—	Aucune
23	Boucle d'auto-supervision Drain3 sur liste blanche (exclusion <code>service_name=drain3</code>)	~30 min	—	Aucune
24	Dette doc — rafraîchissement <code>monitoring-project/CLAUDE.md</code> + PNG d'architecture périmés remplacés.	1-2 h	—	Aucune

Ordre d'exécution suggéré pour la semaine 1 : #1 → #2 → #3 → #4 + #8 (en couple). Les éléments #1 + #2 + #8 ensemble représentent à peu près huit heures d'effort et apportent la plus grande amélioration de qualité perçue. Les éléments #3 + #4 sont le travail « la prose est désormais honnête ». Les éléments #5–#11 ferment les chemins d'escalade faux positifs. Les éléments #12–#24 forment la deuxième vague.

11.2 Livré avant la clôture du jour 1 (huit user stories)

Les huit user stories ci-dessous sont livrées sur le dépôt du service de tri avec la suite de tests à 296/296 au vert et le conteneur de tri redéployé sur `observability-gpu-uswest2` à 09h11 UTC le jour 1. Chaque entrée porte l'identifiant de *commit*, les tests qui la soutiennent et la preuve d'acceptation qui l'a refermée.

11.2.1 D24 + D25 — Migration GPU *cloud* (livrée 2026-05-21)

La migration GPU *cloud* vers `us-west-2` (`g5.xlarge` A10G 24 Go, hôte pour `qwen2.5:14b-instruct`) figurait comme candidate sprint 4 depuis le 2026-04-27, date d'approbation du quota `G+VT vCPU on-demand`. La décision a été **abandonnée pour des raisons de coût le 2026-05-19**, puis **annulée et livrée le 2026-05-21** après une reconstruction du cadre de coût. Les deux moitiés de la décision sont consignées ici parce que le revirement est lui-même l'histoire intéressante.

L'abandon du 2026-05-19. Une `g5.xlarge` coûte environ un dollar américain par heure en *on-demand*, soit grossièrement 720 \$ par mois si laissée en marche, avec un arrêt automatique agressif ramenant le coût mensuel réaliste à 200–300 \$. Sous le cadre de référence 24/7, c'est un ordre de grandeur de plus que le *runner* portable GTX 1060 + `qwen2.5:7b`, qui avait été le *runner* de production pendant vingt-sept jours sans régression de qualité mise au jour dans les audits quotidiens ou le scoring RCA de la fenêtre d'investigation. Sous ce cadre, la migration était indéfendable.

Le revirement du 2026-05-21. Deux jours plus tard, le calcul de coût a été reconstruit autour d'une passerelle Lambda d'extinction automatique et la réponse a basculé. La forme d'usage réelle est *bursty* (un webhook toutes les ~15 minutes pendant un *flap*, inactif le reste du temps); 24/7 était la mauvaise référence. Une Lambda surveillant le trafic inactif arrête l'instance après 20 minutes sans webhooks; SQS bufférise le démarrage à froid (~90 s de réveil au premier webhook d'une fenêtre calme); une alarme de facturation CloudWatch plafonne durement le compte à 50 \$/mois. Cycle d'usage réaliste inférieur à 15 %, faisant passer le coût effectif *on-demand* de 730 \$ à ~100 \$/mois. Migration livrée le jour même comme documenté dans l'entrée D24 du journal des décisions : instance `i-03aec662b1a47ad8f`, nom d'hôte `Tailscale observability-gpu-uswest2`, `qwen2.5:14b-instruct` en primaire avec le *runner* portable conservé en repli (bascule au niveau DNS sur la cible webhook de Grafana). Le test de fumée phase 2 sur le nouveau *runner* a produit une médiane de bout en bout de ~38 s contre ~5 min sur la référence portable — une accélération de ~8× sans régression comportementale. Un verrouillage d'accès co-livré (D25) achemine le webhook Grafana via une API Gateway en liste blanche d'IP, restreint les chemins opérateur à Tailscale et désactive SSH public sur l'instance.

Effets de bord. Premièrement, le *bundle* de latence P0 original du sprint 4 (instrumentation par étape, *timeout* dur 180 s du pipeline, concurrence sérielle Ollama, réécriture en *native tool-calling* MCP) est livré comme marge défensive plutôt que comme correction de latence — la migration a traité le plafond sous-jacent « LLM unique sur GPU grand public » que le *bundle* cherchait à contourner. Deuxièmement, l'approbation de quota G+VT `us-west-2` (4 vCPU, octroyée le 2026-04-27) est désormais utilisée plutôt qu'en réserve. Troisièmement, la leçon méthodologique : les abandons pour raisons de coût doivent expliciter l'hypothèse de cycle d'usage; la décision du 2026-05-19 était correcte sous un cadre 24/7 et fausse sous un cadre de cycle d'usage *bursty*, et le cadre était l'hypothèse porteuse.

11.2.2 SF-6 — Refonte du courriel (livrée 2026-05-23, commit `f6d0a1b`)

Le courriel d'escalade a été réécrit de bout en bout pour une forme lisible par l'opérateur : un sujet de la forme `[env] [ns] [VERDICT] alertPlain` (avec un plafond de 70 caractères imposé en test), une bannière, des pastilles d'identité, un WHAT en une ligne, une narration WHY, une unique ACTION recommandée, quatre boutons opérateur explicites (acquitter, escalader davantage, rejeter, commenter) et un rendu CSS *inline* compatible courriel afin que le message s'affiche correctement sur Gmail web, Outlook web et les clients mobiles courants. Onze nouveaux tests assertent la contrainte de sujet, la présence des boutons et l'échappement CSS *inline*. La suite de tests est passée de 274 à 285.

11.2.3 SF-7 — Page de retour d'opérateur (livrée 2026-05-23, commit `10ebd4e`)

Les quatre boutons du courriel alimentent une nouvelle page `rate.html` servie par le service de tri à `/rate/{short_id}`; l'opérateur sélectionne une notation et écrit facultativement une `actual_root_cause` libre et un commentaire. Les soumissions atteignent un nouvel endpoint `POST /feedback/rate/{short_id}` adossé à une migration de schéma à sept colonnes sur la table de retour US-5.3 existante (colonnes `rating`, `comment`, `rated_at`, identifiant du noteur ajoutées aux côtés des champs `override/confirm` existants). L'endpoint est idempotent sur `short_id`. Avec SF-6, cela ferme la boucle de retour depuis la surface courriel jusqu'à la table `feedback` qui alimente les métriques `trriage_precision` et `trriage_recall` (D20).

11.2.4 SF-4 — Regroupement par même alerte sur le tableau de bord (livré 2026-05-23, commit 7d0adeb)

Le tableau de bord phase 2 (`/dashboard/v2`) affichait auparavant l'historique RCA brut, une ligne par déclenchement, de sorte qu'une alerte flottante remplissait toute la surface. SF-4 regroupe par empreinte au rendu (pas de migration de schéma), conserve la RCA la plus récente par empreinte et pagine la liste unique. L'impact en direct le jour 1 : 288 lignes brutes se réduisent à 26 empreintes uniques dans la fenêtre par défaut de 24 heures. Le travail de déduplication côté schéma (DA-5, ci-dessous) est livré comme correction structurelle complémentaire.

11.2.5 Phase 2.1 — Route de la page de détail (livrée 2026-05-22, commits 9b61e74 + a904eda)

La route `/dashboard/v2/alert/{short_id}` rend une décision unique en pleine page, avec un panneau de gauche montrant `cause`, `action` et le contexte de l'historique de l'incident, et une barre latérale droite montrant la liste des preuves, les modèles Drain3 qui se sont déclenchés, le fil de raisonnement du LLM et la liste des alertes apparentées dans la fenêtre de corrélation. Cette page est la surface sur laquelle l'opérateur clique depuis le bouton acquitter du courriel ; c'est la vue d'audit en lecture seule par l'opérateur du raisonnement de la plateforme.

11.2.6 DA-5 + DA-4 — Déduplication par famille d'alertes + empreintes drain3 par hachage de contenu (livré 2026-05-25, commit b8fa2c8)

DA-5 introduit une carte `ALERT_FAMILIES` couvrant les quatre classes d'alertes dominantes (`cpu`, `memory`, `disk`, `loki-disk`) et un auxiliaire `family_dedup_key(alert)`, de sorte que `MediumCpuUsage`, `HighCpuUsage` et `CriticalCpuUsage` se déclenchant sur la même instance se regroupent comme une seule famille dans la couche de suppression plutôt que comme trois empreintes distinctes en course l'une avec l'autre. Les *alerthnames* inconnus retombent sur l'empreinte Grafana ; les cas sans instance retombent sur un regroupement limité au service. DA-4 durcit la forme d'empreinte Drain3 en `drain3-{service}-{sha256(top-3-templates)[:12]}`, avec repli sur `anomalous_lines` puis sur l'ancienne clé en clair, de sorte que les lots Drain3 sans rapport ne se regroupent plus silencieusement dans une même fenêtre de déduplication. Onze nouveaux tests couvrent la stabilité de hachage, la divergence sur des modèles différents, la séparation par service, le comportement de repli et le regroupement de bout en bout High→Critical CPU avec `decision_id` lié. La suite de tests est passée de 285 à 296.

11.2.7 Observation de livraison du jour 1

Les huit user stories ci-dessus étaient initialement classées sur les deux semaines du plan jour par jour, et ont été achevées le jour 1. Ce n'est pas accidentel : la clôture du sprint 3 a laissé les surfaces suite, tableau de bord et notificateur dans un état où de petits *commits* bien périmétrés pouvaient atterrir à haute vitesse, et la migration GPU a libéré assez de marge de latence pour que l'opérateur puisse tester les changements en secondes plutôt qu'en minutes. Le travail restant en file d'attente ci-dessous est donc proportionnellement plus important par élément (la cohérence inter-lignes DA-3 et la route `/dashboard/v2/kpi` sont toutes deux à l'échelle de la journée, pas de l'heure).

11.3 En file d'attente pour le reste du sprint 4

Les cinq éléments programmés entre le jour 2 et la clôture du sprint sont listés ci-dessous par ordre calendaire. Chacun est dimensionné en points d'histoire ou en heures selon le cas ; la source de référence du plan de sprint est `monitoring-docs/sprint4-status.html` §14.

- **Mardi 2026-05-26 — DA-2 retrait des actions non sûres (~3 SP).** Une barrière indépendante du bridage qui retire les actions `kubect1` / `ssh` / `restart` / `systemctl` lorsque `rca_quality=actionable` ET qu'aucune cause nommée n'est présente dans la prose, quelle que soit la confiance LLM. Le cas motivant est la suggestion `systemctl restart k3s-node.service` consignée dans la ligne d'audit §7c.4.
- **Mercredi 2026-05-27 — Route tableau de bord phase 3.A.KPI (~3 SP).** Une nouvelle route `/dashboard/v2/kpi` faisant émerger les métriques `triage_precision` / `triage_recall` / taux de suppression / latence moyenne RCA sous forme de cartes de style Grafana au-dessus du tableau d'empreintes uniques. Demandé par le superviseur.
- **Jeudi 2026-05-28 — DA-3 cohérence verdict inter-lignes (~5 SP).** Lorsque la même empreinte se déclenche dans les N minutes d'une décision antérieure, la **cause** antérieure est préfixée dans le contexte LLM avec une règle explicite de cadrage « j'ai changé d'avis parce que... ». Empêche la plateforme d'émettre des RCA contradictoires sur des déclenchements consécutifs d'un même *flap*.
- **Vendredi 2026-05-29 — Persistance des filtres URL + câblage du filtre de plage (~3 SP).** Les filtres du tableau de bord (verdict, sévérité, famille d'*alertname*, plage temporelle) sont encodés dans la chaîne de requête URL afin que l'opérateur puisse marquer et partager des vues filtrées, et que le bouton retour restaure le contexte antérieur. Le câblage du filtre de plage achève la couche de schéma du jour 1 laissée à mi-chemin.
- **Vendredi S2 (étirement) — SF-5 modificateur de verdict soutenu vs. pic (~3 SP).** Ajoute une caractéristique « durée au-dessus du seuil » à la charge utile d'alerte et classe les franchissements de courte durée comme `rca_quality=transient_spike` avec `action_taken=shelved`. Objectif d'étirement — ne sera livré que si DA-3 atterrit proprement et que le volume de retour d'opérateur issu de SF-7 soutient le réglage du seuil.

Les cinq éléments totalisent ensemble environ dix-sept points d'histoire sur quatre journées de travail plus un étirement. À la clôture du sprint 4 le 2026-06-05, la suite est attendue à ~310 tests au vert ; le prochain audit critique RCA (une répétition à 8 critères de l'audit du 2026-05-21 consigné au §9.3) est programmé pour le lundi suivant.

11.4 Tier 3, RAG curé et palier résilience (reportés au sprint 5)

Les trois éléments planifiés plus lourds ci-dessous n'ont *pas* été retenus pour le sprint 4 sur le cadrage de maturité opérationnelle ; ils figurent dans le périmètre du sprint 5 (chapitre 12) et sont résumés ici brièvement seulement afin que le lecteur du sprint 4 n'en perde pas la trace.

Tier 3 — collecte agentique itérative. La conception Tier-3 remplace la reprise à action contenue à coup unique par une boucle d'appel d'outils sur trois itérations, chaque itération produisant un arbre d'hypothèses partiel que le modèle étoffe à la suivante. L'arbre d'hypothèses est stocké dans le *prompt* comme section structurée ; à chaque itération, le modèle est invité à étendre la feuille la plus prometteuse, à collecter les preuves de soutien via un unique appel MCP et à mettre à jour l'arbre. Conditions de terminaison : seuil de confiance atteint, limite de profondeur maximale atteinte ou budget de latence total épuisé. C'est matériellement plus permissif que la reprise à action contenue ; le conditionnement à S3-HF-09 (désormais satisfait) reflète la position du projet selon laquelle livrer Tier 3 sans barrière automatisée de régression de qualité serait imprudent.

Récupération RAG curée (US-3.3 / US-3.4). Une fois que la table de retour d’opérateur de l’Épopée 2 aura accumulé suffisamment d’entrées notées via la surface SF-7, la récupération curée devient digne de l’effort d’ingénierie. La conception : un job hebdomadaire lit la table de retour pour les sept derniers jours ; les entrées au-dessus d’un seuil de qualité sont écrites en YAML dans un fichier `library_curated.yaml` ; la section exemplaire du *prompt* de tri est complétée par un petit nombre ($k = 3$ positifs, $k = 2$ anti-exemples) récupérés par BM25 sur ce corpus curé. Le *dry-run* de la fenêtre d’investigation a établi que BM25 sur la bibliothèque *de base* de quatorze archétypes est dégénéré ; le scénario de production est BM25 sur une bibliothèque enrichie, et c’est ce qui sera mesuré une fois que SF-7 aura alimenté assez de lignes notées.

Palier résilience (R-1 à R-5). La fenêtre d’investigation de 20 jours a fait émerger cinq éléments de résilience opérationnelle, retenus pour inclusion au sprint 5 :

- **R-1** Métrique de *heartbeat* cron en condition réelle sur une *push-gateway* Prometheus avec une règle Grafana `LiveAuditMissing`.
- **R-2** Déplacer le cron d’audit en condition réelle depuis l’éphémère `claudie-controller` vers le pérenne `observability-rca-monitoring` EC2 via un *timer* systemd.
- **R-3** Lint CI qui échoue si une date de fin de fenêtre dans un `sprint*-plan.md` quelconque est passée.
- **R-4** Badge d’obsolescence `the-bigger-picture.md` dans le README du dépôt, rouge à quatorze jours.
- **R-5** Sauvegarde S3 de `SESSION_HANDOFF.md`, intégrée dans le travail de sauvegarde nocturne O-10.

11.5 Ancre de section héritée pour la migration GPU

L’histoire combinée abandon-puis-livraison de la migration GPU (initialement consignée sous cette étiquette comme travail futur du sprint 4) vit désormais au §11.2.1 sur la liste des user stories livrées. Cette ancre est conservée afin que la puce périmètre-et-limites du chapitre 1 et la référence croisée exigences-non-fonctionnelles du chapitre 3 continuent de se résoudre.

CHAPITRE 12

Sprint 5 — Planifié (2026-06-06 au 2026-06-17)

Le sprint 5 court du 2026-06-06 au 2026-06-17 et déplace le centre de gravité de la plateforme vers l'extérieur. Les sprints 1 à 4 ont construit et durci la boucle de tri centrale contre une unique pile web représentative (Spring Boot + Kong + MySQL + React, plus l'application de test `car-rental` arrivée pendant la clôture du sprint 3). Le mandat du sprint 5 est double : déployer un banc de test microservice canonique qui exerce correctement les chemins multi-applicatifs de la plateforme et livrer le travail de configuration opérateur et de durcissement production qui faisait la file d'attente à travers les sprints 3 et 4.

Le sprint est structuré en trois épopées concurrentes. Chaque épopée porte assez de travail pour remplir à elle seule le sprint de deux semaines ; la capacité à trois ingénieurs de l'équipe signifie que chaque épopée reçoit un propriétaire disproportionné et deux relecteurs, avec une mise en binôme explicite aux points d'intégration (scénario d'acceptation du banc microservice × seuils Drain3 hiérarchiques, table d'entités d'incident × onglet incidents phase 3.B).

La figure 12.1 situe le sprint 5 dans la chronologie plus large des sprints — la découverte, les sprints de fondation, le sprint 4 en cours et les perspectives sprint 6+ qui closent ce chapitre.

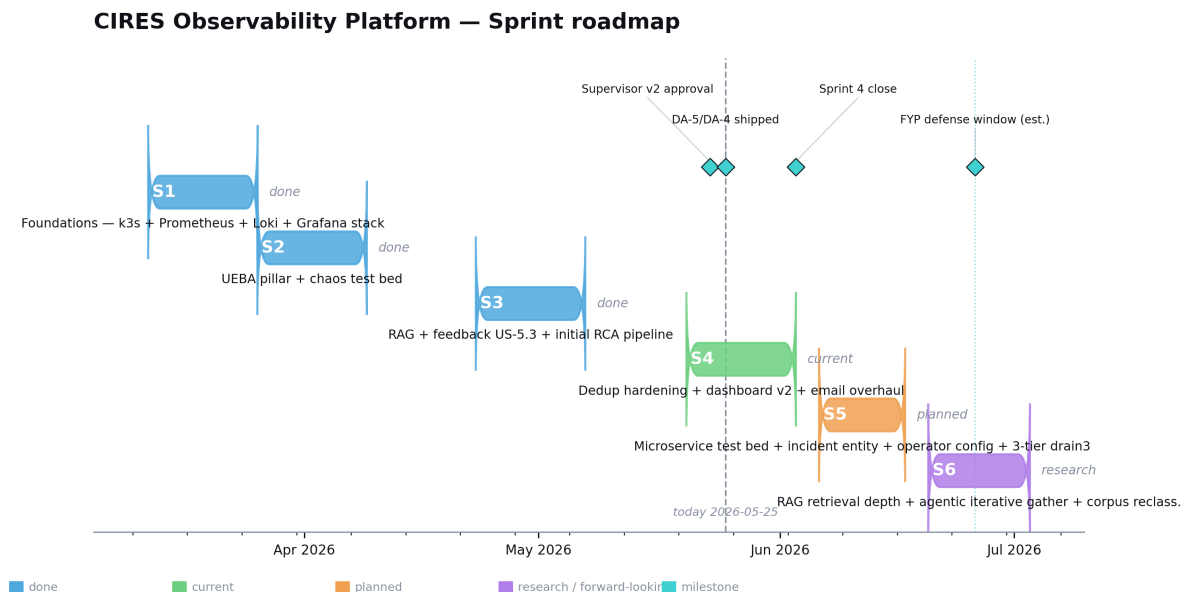


Figure 12.1. Feuille de route des sprints, du sprint 0 au sprint 6+. Chaque colonne porte la fenêtre du sprint et son résultat phare ; les jalons sous la chronologie sont les temps structurels du projet (énoncé du problème, fondation d'observabilité, MVP IA, pare-feu anti-hallucination, migration GPU, banc microservice, Drain3 hiérarchique). Le sprint 4 est ombré comme *en cours*.

12.1 EPIC13 — Banc de test microservice + scénario canonique

Le banc de test de la PoC porte deux applications depuis la clôture du sprint 3 : le dérivé original `react-springboot-mysql` et la seconde application `car-rental`. Deux applications à deux composants chacune suffisent à démontrer que les chemins multi-applicatifs de la plateforme

existent, mais pas à exercer le travail d'agrégation inter-applications que EPIC14 fera émerger. EPIC13 ferme cet écart :

- **SF-13 (clé de voûte, ~8 SP).** Déployer un banc microservice canonique à cinq services sur le cluster k3s existant (`observability-rca-k3s`) : un *front-end*, deux services de *back-end* avec leurs propres bases de données, une file et un *worker* qui consomme la file. Chaque service est instrumenté OTel, scrapé par Prometheus, et ses journaux de conteneur sont expédiés vers Loki par le récepteur `filelog/containers` de l'OTel Collector. Les *Helm charts* vivent dans le dépôt `monitoring-project` ; le banc est reproductible depuis k3s vierge via un seul lot de `helm install`.
- **SF-9 (~3 SP).** Un nouveau pont `k8s-mcp` exposant à l'étape de collecte de contexte du pipeline de tri des outils en lecture seule `kubectl get`, `kubectl describe` et de l'API Rancher. Ferme l'écart transversal selon lequel les alertes au niveau pod ne pouvaient être que partiellement diagnostiquées parce que le LLM ne voyait jamais la sortie `kubectl describe` du pod lui-même.
- **SF-10 (~3 SP).** Supervision au niveau applicatif (p95 par endpoint, taux d'erreurs par endpoint, saturation du pool de connexions par service) comme un ensemble de nouvelles règles Grafana alimentant les services du banc microservice.
- **SF-11 (~3 SP).** Validation de scénario : l'induction de chaos « un développeur supprime une lettre dans une clé de configuration » qui produit une défaillance en cascade à travers le banc microservice et exerce les chemins de corrélation inter-applications de la plateforme. Le critère d'acceptation est un seul courriel RCA cohérent nommant l'édition de la clé de configuration, et non cinq courriels d'alerte distincts.

EPIC13 est le prérequis de tout ce qui est dans EPIC14 et de l'élément de conception S5-DRN-01 du §12.4. Les seuils Drain3 de Tier 2 (par application) et de Tier 3 (à l'échelle du système) ne peuvent être significativement validés tant que le banc n'a pas au moins trois composants par application.

12.2 EPIC14 — Entité incident + surfaces d'insights barre latérale

L'unité *alerte* actuelle du tableau de bord est trop petite pour les incidents qui s'étendent sur plusieurs alertes. EPIC14 introduit une entité **incident** au-dessus des lignes de décision existantes :

- **Table d'incidents phase 2.2.** Une table `incidents` au niveau schéma agrégeant les décisions apparentées par appartenance à la fenêtre de corrélation et par service / instance partagés. Le tableau de bord rend une carte d'incident par incident, avec les alertes constitutives en *drill-down*.
- **Statistiques phase 3.A.** Une bande KPI montrant le taux d'incidents sur les 7/30 derniers jours, le temps moyen jusqu'à la RCA, le rapport rejet-vs-escalade et le taux de correction d'opérateur par famille d'alerte. Étend la route phase 3.A.KPI qui atterrit pendant le sprint 4.
- **Anomalies phase 3.A.** Un onglet signaux détectifs rendant le taux de nouveauté Drain3 par service contre la ligne de base même-heure-il-y-a-7-jours, avec une tuile par service. Surface opérateur pour les données que le *poller* Drain3 produit déjà.

- **Incidents phase 3.B.** La vue chronologique complète d'un incident — un incident, ses alertes constitutives, le fil de réponse de l'opérateur et la boucle de retour post-incident. C'est la surface qui complète le chemin d'ingestion du retour SF-7 : un opérateur peut noter la gestion par la plateforme d'un incident entier, et non seulement d'une alerte isolée.

Les deux points d'intégration transversaux sont l'onglet Anomalies de la phase 3.A (qui dépend du travail Drain3 hiérarchique du §12.4) et le modèle de données de la table d'incidents (que le scénario du banc microservice SF-13 exercera comme test d'acceptation).

12.3 EPIC15 — Configuration opérateur + durcissement production

- **Phase 3.C Alerts-config.** Une page côté opérateur pour éditer les seuils d'alerte Grafana, les opt-in de barrière de récurrence et les paramètres de ligne de base adaptative depuis le tableau de bord de tri. Écrit en retour vers Grafana via l'API des règles provisionnées.
- **Phase 3.C Drain3-engine.** Surface opérateur pour les mineurs Drain par service — instantané, restauration et réglage des seuils par service, exposant la tuyauterie S3 dormante enfin câblée pendant ce sprint.
- **Phase 3.C Integrations.** Cibles de notification Slack et webhook aux côtés du chemin d'escalade SMTP existant. Clés de routage par équipe pilotées par étiquette de service.
- **Phase 4 SSE + filtre côté serveur.** Mises à jour *push* du tableau de bord via *server-sent events* ; évaluation du filtre côté serveur de sorte que le tableau de bord rende uniquement la vue d'intérêt de l'opérateur. Remplace le filtrage côté client actuel, qui s'adapte mal au-delà d'un corpus de mille lignes.
- **Phase 4 identité opérateur.** Identité par opérateur via ACL Tailscale. Les notations de retour SF-7 portent déjà une colonne **rater** ; voici la tuyauterie qui la peuple.
- **Phase 4 pré-compilation React.** Remplacer le HTML rendu actuellement par Jinja2 par un *bundle* React pré-compilé pour les surfaces les plus lourdes du tableau de bord.
- **SF-12 + DA-6.** *Drill-down* RCA au niveau pod — cliquer à travers une alerte au niveau pod ouvre en *inline* les événements du pod, les journaux de conteneur et les commandes de diagnostic lisibles en *exec shell*.

12.4 Nouvel élément de conception — S5-DRN-01 seuils Drain3 hiérarchiques

La discussion de conception tenue aux côtés de la livraison DA-5 / DA-4 le 2026-05-25 a fait émerger un manque architectural dans la couche de détection d'anomalies du projet. Le *poller* Drain3 courant déclenche un webhook `Drain3AnomalyDetected` en utilisant un unique seuil par service sur le taux de modèles nouveaux. Ce seuil par service est le maillon faible de la chaîne de détection : il ne peut distinguer « ce service est bizarre tout seul » de « beaucoup de services sont discrètement bizarres en même temps ». Une dérive en dessous du seuil mais corrélée à travers plusieurs composants ressemble à du silence vu de la barrière par service, alors même que le système dans son ensemble a clairement bougé.

La conception à trois paliers. La solution sprint 5 proposée introduit trois paliers de seuil d'anomalie se déclenchant indépendamment, chacun scopé à un niveau différent de la hiérarchie du système. Le déclenchement de Tier N ne supprime pas Tier $N + 1$ — ce sont des signaux complémentaires, pas une chaîne de repli.

- **Tier 1 — composant.** Le plus étroit : par *backend* / *frontend* / capteur / db de chaque application. Calculé au périmètre le plus fin par lequel Drain3 peut regrouper. Attrape la dérive d'un seul composant — « le pod `checkout-backend` génère des modèles nouveaux à un taux inhabituel spécifiquement pour ce composant. »
- **Tier 2 — application.** Par application, agrégeant le taux d'anomalies à travers les composants de cette application (*backend* + *frontend* + db + capteur en un seul seau). Attrape la dérive inter-composants au sein d'une application même quand aucun composant unique ne franchit Tier 1.
- **Tier 3 — système.** Agrégé à travers toutes les applications sous supervision. Attrape la dérive à l'échelle de la plateforme qu'aucune application unique ne remarquerait isolément — le cas « beaucoup de services sont discrètement bizarres » auquel la barrière par service est structurellement aveugle.

Hierarchical drain3 anomaly thresholds (Sprint 5 design)

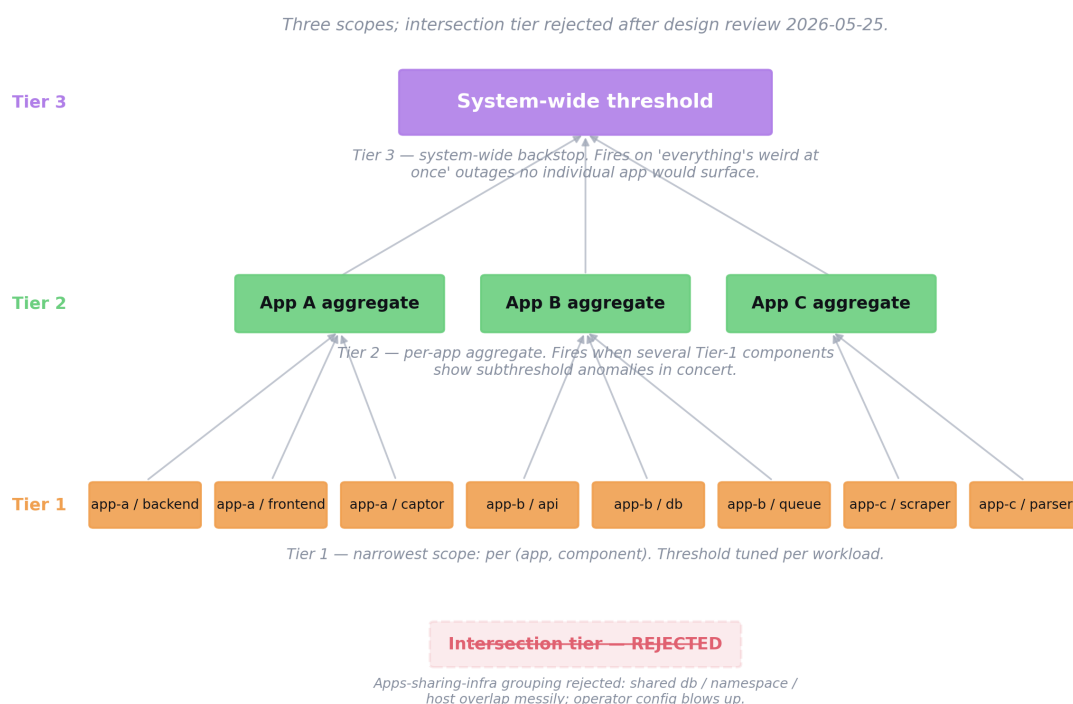


Figure 12.2. La conception sprint 5 des seuils Drain3 hiérarchiques — trois paliers (composant, application, système) qui se déclenchent indépendamment les uns des autres. Le déclenchement de Tier N ne supprime pas Tier $N + 1$. Le panneau de droite annote le palier « intersection » rejeté (applications partageant une infrastructure), abandonné comme combinatoirement flou.

Rejeté : le palier intersection. L’esquisse originale incluait un quatrième palier d’applications partageant une infrastructure (db, *namespace* ou hôte). Il a été abandonné comme combinatoirement flou : les définitions d’intersection se chevauchent confusément et leur exposition comme regroupements réglables fait exploser la surface de configuration opérateur pour un signal limité que Tier 3 couvre déjà comme filet de sécurité.

Dépendance de validation. La validation requiert que SF-13 (banc microservice) soit en service avec au moins deux applications portant chacune trois composants ou plus, afin que les chemins d’agrégation inter-applications puissent être exercés. Le scénario d’acceptation est une induction de chaos inter-applications où aucun composant individuel ne franchit Tier 1 (le taux de nouveauté de chaque composant reste dans sa propre ligne de base) mais où l’agrégat Tier 2 d’une application — ou l’agrégat Tier 3 à l’échelle système — se déclenche bien. Suivi dans Jira comme **S5-DRN-01** (id 320) parenté à EPIC13 ; documenté dans [monitoring-docs/solution-brief.html §13.7](#).

12.5 Forme de déploiement production — OTel à agent unique pour le NOC

Une seconde discussion prospective du 2026-05-25 concerne la forme de la pile de collecte par hôte au passage du PoC à la production CIREs — élément de transition, pas livrable de sprint, consigné ici pour que la conversation d’autorisation se tienne avec l’alternative déjà spécifiée. Chaque hôte du PoC fait tourner deux démons : l’OTel Collector (journaux via `filelog/containers`, traces via `otlp`) et `node-exporter` pour les métriques hôte. La production CIREs impose une contrainte : l’équipe NOC est réticente à installer de nouveaux *exporters* en production, chaque binaire étant une surface de menace supplémentaire. CIREs exploite déjà une collecte métriques mature basée sur **Zabbix** ; la plateforme ne la remplacera pas et consomme ces métriques via les *exporters* existants.

La proposition à agent unique. L’OTel Collector peut absorber les métriques aussi, éliminant `node-exporter` : trois récepteurs natifs s’en chargent — `hostmetrics` (CPU, mémoire, disque, réseau depuis `/proc` et `/sys`, sans `CAP_*`), `prometheus` (scrape des endpoints `/metrics` applicatifs existants) et `kubeletstats` (k8s uniquement). Le résultat est un binaire par hôte : un *systemd unit*, un fichier YAML déclaratif et une sortie exclusivement sortante via OTLP forment une cible d’audit plus simple que N *exporters*, et OpenTelemetry est un projet CNCF *graduated* avec une chaîne d’approvisionnement plus solide. La contrepartie est l’empreinte mémoire (~80–150 MB *resident* contre ~15 MB, acceptable en production) et le fait que quelques métriques privilégiées (processus, BPF) seraient abandonnées si le NOC refuse les capacités requises. La migration Promtail-vers-OTel du sprint 1 fixait le précédent : le Collector absorbe plus, pas moins. La décision finale reste pilotée par le NOC.

12.6 Perspectives sprint 6+

Au-delà du sprint 5 se trouvent trois directions à plus long terme qui méritent d’être consignées pour la considération du superviseur, mais qui ne sont pas encore inscrites en sprint. Ce sont les continuations naturelles des travaux décrits aux chapitres 7 et 11.

- **Un modèle de déploiement multi-tenant.** La plateforme actuelle est mono-tenant par construction ; un déploiement multi-tenant (un service de tri par unité métier, ponts MCP et pile d’observabilité partagés) requerrait isolation par *namespace* de l’historique

RCA, bibliothèques d'exemplaires et calibrage de confiance par tenant. Périmètre naturel du sprint 6+.

- **Intégration directe avec les systèmes de production de Tanger Med.** L'*onboarding* de services de production réels demanderait gestion soigneuse des PII, un programme SLO pour fixer des seuils de confiance significatifs et un contrôle des changements formel pour les mises à jour de *prompt* et d'exemplaires. C'est le chemin vers la transition production avec greenlight superviseur que le mémo de périmètre du projet identifie comme l'état terminal.
- **Évaluation fédérée de modèles.** À mesure que le projet accumule des issues RCA étiquetées via SF-7 et SF-13, comparer des LLM alternatifs sur un échantillon retenu devient faisable : `deepseek-r1:14b-distill`, `mistral-nemo:12b` et `qwen2.5:32b-instruct` aux côtés de l'incumbent. Successeur naturel de la migration GPU D24.
- **Observabilité prédictive (carve-out EPIC12).** L'onglet signaux détectifs S4-PRED-01 absorbé dans EPIC14 Phase 3.A.Anomalies est le pied dans la porte ; PRED-02 (capacité), PRED-03 (saturation), PRED-04 (cascade) sont des candidats sprint 6 qui poussent la plateforme du diagnostic vers le prédictif.

CHAPITRE 13

Conclusion

Ce mémoire a rendu compte de la conception et de la mise en œuvre de la plateforme d’observabilité CIREs, une solution d’observabilité augmentée par IA construite sur quatre sprints (avec un cinquième scapé au chapitre 12) chez CIREs Technologies. Les sprints 0 à 3 sont clôturés ; le sprint 4 est en cours actif avec huit user stories livrées le jour 1 et cinq autres programmées sur le reste du plan. La plateforme répond à l’écart opérationnel identifié pendant la recherche de terrain du sprint 0 — à savoir que l’équipe découvrait les problèmes de production après coup plutôt que de manière proactive — en combinant une pile d’observabilité *open source* classique avec une couche de tri par IA qui transforme chaque alerte en une RCA structurée et pré-investigée.

Les contributions principales du travail sont :

1. **Une mise en œuvre de référence de bout en bout.** Pile d’observabilité provisionnée par Terraform, configurée par Ansible, exécutée sur k3s-et-Docker sur AWS, avec un service de tri par IA basé sur FastAPI produisant des RCA structurées à partir des webhooks Grafana via un LLM hébergé localement (`qwen2.5:14b-instruct` sur hôte GPU AWS `g5.xlarge` dédié depuis le 2026-05-21, `qwen2.5:7b` sur portable en repli). Reproductible depuis une infrastructure vierge.
2. **L’invariant d’accès aux données via MCP uniquement.** Règle à l’échelle du projet, en partie architecturale et en partie mécanique (via le lint S3-HF-08), selon laquelle chaque donnée que voit le LLM doit transiter par un pont MCP. Produit un régime de provenance sur l’entrée du LLM, appliquée par les métriques de la couche pont, les rapports d’audit et un lint d’intégration continue.
3. **Le pare-feu anti-hallucination.** Combinaison en couches de mécanismes (exemplaires, liste noire, validateurs de surface et de menu, cause-preuves, bridage de confiance, reprise à action contenue), chacun répondant à un mode de défaillance observé et consigné dans le journal des décisions.
4. **La couche de retour d’information de l’opérateur en boucle fermée.** Schéma et surface capturant les corrections et notations d’opérateur sur les RCA (livrés via SF-6 et SF-7 le jour 1 du sprint 4), propagés dans la couche de suppression, les métriques de précision/rappel et (au sprint 5) le corpus de récupération curé.
5. **Un journal des décisions documenté.** Vingt-cinq décisions durables de conception (D1–D25) consignées de manière contemporaine, comprenant le problème observé, les contraintes, la décision et la vérification.

La plateforme est en exploitation active sur AWS avec des audits quotidiens documentant la santé opérationnelle : latence médiane ~ 38 s sur hôte GPU (depuis ~ 5 min sur portable), suite de tests 296/296 au vert, lint MCP-seul propre depuis S3-HF-08, hôte GPU sans interruption depuis la migration du 2026-05-21. Le travail du sprint 5 capturé au chapitre 12 déterminera si la plateforme est prête pour la transition production avec greenlight superviseur qui constitue l’état terminal de ce projet.

CHAPITRE A

Inventaire des dépôts

Dépôt	Objet
monitoring-project	Rôles Ansible, <i>playbooks</i> , banc de chaos
provisioning-monitoring-infra	État AWS Terraform
monitoring-docs	Le présent document, journal des décisions, page d'architecture, historique des sp
monitoring-triage-service	Service de tri FastAPI, pipeline, validateurs
monitoring-mcp-servers	Les cinq ponts MCP
monitoring-audit-results	Privé — audits quotidiens statiques + en condition réelle
jira	Exports CSV Jira et scripts d'import

Table A.1. Disposition des dépôts du projet (sept dépôts).

CHAPITRE B

Glossaire

AIOps Étiquette informelle pour l'application des techniques d'intelligence artificielle aux problèmes d'exploitation informatique — détection d'anomalies, analyse des causes racines, résumé d'alertes.

Bounded-agency retry (reprise à action contenue) La couche de reprise à un seul appel MCP de la plateforme. Le LLM se voit octroyer exactement un appel d'outil supplémentaire après une violation de validateur ; le schéma et la chaîne de retour contraignent l'appel.

Cause-evidence rule (règle cause-preuves) Une règle de validateur qui tokenise la prose RCA et les preuves citées avec `[a-z]{4,}` et exige un recouvrement hors-mots-vides. Rejette les RCA dont la cause n'est pas étayée par les preuves citées.

Confidence clamp (bridage de confiance) L'étape de pipeline qui force `confidence` $\leq 0,4$ si les validateurs de surface ou de menu d'hypothèses se sont déclenchés, si la qualité RCA vaut `data_starved` ou si les actions paraissent modélisées. Retire les `suggested_actions` quand il s'enclenche et émet à la place des `diagnostic_steps` en lecture seule.

Drain3 Un algorithme de regroupement de modèles de journaux (variante paramétrable de Drain). Utilisé dans la plateforme comme quatrième pilier de télémétrie faisant émerger les modèles nouveaux.

Exemplar archetype (archétype d'exemplaire) L'un des quatorze exemples de RCA rédigés à la main injectés dans le *prompt* comme échafaudage structurel avant le bloc de preuves.

Grafana unified alerting Le successeur d'Alertmanager dans la pile Grafana. Toutes les règles d'alerte et points de contact vivent à l'intérieur même de Grafana.

Hallucination Une affirmation produite par un LLM qui n'est pas entaillée par le contexte d'entrée (p. ex. un service qui n'existe pas ; une valeur métrique qui n'était pas dans le bloc de preuves).

Hypothesis menu (menu d'hypothèses) Une RCA qui hésite avec plusieurs causes possibles. Détecté par le validateur S3-HF-03.

MCP Model-Context-Protocol — spécification ouverte pour connecter les applications LLM à des sources de données et outils externes.

MCP-only invariant (invariant MCP-seul) La règle à l'échelle du projet selon laquelle chaque donnée que voit le LLM doit transiter par un serveur MCP.

node-exporter L'exporter Prometheus standard pour les métriques d'hôte Unix (CPU, mémoire, disque, système de fichiers, réseau).

OTel OpenTelemetry, le standard ouvert de fait pour l'instrumentation de télémétrie.

RCA Root-Cause Analysis (analyse des causes racines) — l'explication structurée des raisons pour lesquelles un incident est survenu.

SRE Site Reliability Engineering — la discipline opérationnelle formalisée par l’ouvrage et le *workbook* homonymes de Google.

Surface-only LEDE (ouverture de surface) Une ouverture de RCA qui reformule l’alerte (p. ex. en commençant par l’expression PromQL ou la valeur observée) au lieu de nommer une cause. Détectée par l’ensemble de regex `_SURFACE_ONLY_LEDE_PATTERNS`.

Tailscale tailnet Le maillage basé sur WireGuard qui connecte tous les hôtes de la plateforme. MagicDNS fournit des noms d’hôte stables.

Triage (tri) Dans le vocabulaire de la plateforme, la couche IA qui consomme les alertes et produit des RCA (à distinguer de la pratique humaine de *triage* en réponse aux incidents).

CHAPITRE C

Résumé de la chronologie des sprints

Sprint	Fenêtre	Résultat phare
0	Février 2026 – début mars 2026	Énoncé du problème cristallisé ; hypothèse de solution (observabilité + tri par IA) choisie.
1	2026-03-10 – 2026-03-30	Pile d’observabilité sur site à trois VM déployable en 15 min via Ansible.
2	2026-03-31 – 2026-04-23	Migration AWS, k3s pour la charge applicative, MVP RCA par IA (5 serveurs MCP + tri FastAPI + Ollama).
2-ext	2026-04-23 – 2026-04-29	Épopée 5 : Drain3 par service, lignes de base d’entités, retour d’information en boucle fermée, barrières de récurrence, seuils adaptatifs.
3	2026-04-23 – 2026-05-19	Clôture qualité RCA, épopée pare-feu anti-hallucination ajoutée le 2026-04-29 après 0b215ef3, fenêtre d’investigation de 20 jours du 04-30 au 05-19.
4	2026-05-25 – 2026-06-05	<i>En cours.</i> Livraison jour 1 de la migration GPU (D24/D25), refonte du courriel SF-6, page de retour d’opérateur SF-7, regroupement tableau de bord SF-4, page détail phase 2.1, déduplication DA-4/DA-5 ; en file d’attente : DA-2, phase 3.A.KPI, DA-3, filtres URL, SF-5 en étirement. Suite 296/296 au vert.
5	2026-06-06 – 2026-06-17	<i>Planifié.</i> EPIC13 banc de test microservice (SF-13) + scénario canonique ; EPIC14 entité incident et surfaces phase 3.A/3.B en barre latérale ; EPIC15 configuration opérateur et durcissement phase 4 ; S5-DRN-01 seuils Drain3 hiérarchiques (3 paliers, palier intersection rejeté).
6+	À partir du 2026-06-18	Perspectives : modèle de déploiement multi-tenant, pilote de transition production Tanger Med, évaluation fédérée de modèles, carve-out d’observabilité-prédictive EPIC12 (PRED-02/03/04).

Table C.1. Chronologie des sprints, condensée. Sprints 0–3 clôturés ; sprint 4 en cours le 2026-05-25 ; sprint 5 scopé ; perspectives sprint 6+ consignées.

Références bibliographiques

Bibliographie

- [1] B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.), *Site Reliability Engineering : How Google Runs Production Systems*, O'Reilly, 2016.
- [2] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, S. Thorne (eds.), *The Site Reliability Workbook : Practical Ways to Implement SRE*, O'Reilly, 2018.
- [3] P. He, J. Zhu, Z. Zheng, M. R. Lyu, *Drain : An Online Log Parsing Approach with Fixed Depth Tree*, IEEE International Conference on Web Services (ICWS), 2017.
- [4] OpenTelemetry Project, *OpenTelemetry Specification*, version 1.x, <https://opentelemetry.io/docs/specs/otel/>.
- [5] J. Volz et al., *Prometheus : Up & Running*, O'Reilly, 2018, and <https://prometheus.io/docs/>.
- [6] Grafana Labs, *Grafana Loki documentation*, <https://grafana.com/docs/loki/latest/>.
- [7] Y. Shkuro, *Mastering Distributed Tracing*, Packt, 2019, and <https://www.jaegertracing.io/docs/>.
- [8] Anthropic, *Model Context Protocol — specification*, <https://modelcontextprotocol.io/>.
- [9] Qwen Team, Alibaba Group, *Qwen2.5 Technical Report*, arXiv :2412.15115, 2024.
- [10] Ollama, *Run large language models locally*, <https://ollama.com/>.
- [11] Rancher, *k3s — Lightweight Kubernetes*, <https://k3s.io/>.
- [12] HashiCorp, *Terraform documentation*, <https://developer.hashicorp.com/terraform/docs>.
- [13] Red Hat, *Ansible documentation*, <https://docs.ansible.com/>.
- [14] Tailscale Inc., *Tailscale documentation*, <https://tailscale.com/kb/>.